

Task 1 – DNS Server and Zone Validation

Objective

Validate the DNS configuration and DNS Server service on **DC01**, inspect the existing forward and reverse lookup zones, review DNS resource records, and test name resolution within the **benlab.local** domain.

The purpose of this task was to establish a baseline of the existing DNS infrastructure before making DNS configuration changes.

Environment Validation

The following DC01 configuration was verified:

- **Server Name:** DC01
- **Domain:** benlab.local
- **IPv4 Address:** 10.20.20.10
- **Subnet Mask:** 255.255.255.0
- **Default Gateway:** 10.20.20.254
- **IPv4 DNS Server:** 127.0.0.1
- **DNS Server Role:** Installed and operational

DC01 functions as both an **Active Directory Domain Controller and DNS server** for the BENLAB environment.

PowerShell Commands

DNS client configuration:

```
Get-DnsClientServerAddress -AddressFamily IPv4
```

Complete network configuration:

```
ipconfig /all
```

DNS Server service status:

```
Get-Service DNS
```

DNS zones hosted by DC01:

```
Get-DnsServerZone
```

Resource records contained in the `benlab.local` zone:

```
Get-DnsServerResourceRecord -ZoneName "benlab.local"
```

Forward and Reverse Zone Inspection

The DNS zones hosted by DC01 were inspected using PowerShell and DNS Manager.

Important forward lookup zones included:

- `benlab.local`
- `_msdcs.benlab.local`

The reverse lookup zone for the BENLAB `10.20.20.0/24` network was:

`20.20.10.in-addr.arpa`

The `benlab.local` zone contained several important DNS record types:

- **A** — Hostname to IPv4 address
- **AAAA** — Hostname to IPv6 address
- **NS** — Identifies authoritative DNS servers
- **SOA** — Contains authoritative zone information
- **SRV** — Identifies services such as LDAP, Kerberos, and Global Catalog

Existing A records included:

APP01	→	10.20.20.15
DC01	→	10.20.20.10
FILESRV01	→	10.20.20.50
WIN01	→	10.20.20.140

Resolution Testing

Forward DNS resolution was tested with:

```
Resolve-DnsName dc01.benlab.local
```

```
Resolve-DnsName FILESRV01.benlab.local
```

```
Resolve-DnsName WIN01.benlab.local
```

All three hostnames successfully resolved to their corresponding IPv4 addresses.

Reverse DNS resolution was then tested:

```
Resolve-DnsName 10.20.20.10
```

```
Resolve-DnsName 10.20.20.50
```

```
Resolve-DnsName 10.20.20.140
```

Successful results included:

10.20.20.10 → DC01.benlab.local

10.20.20.50 → FILESRV01.benlab.local

WIN01 Missing PTR Finding

The reverse lookup for WIN01 was tested using:

Resolve-DnsName 10.20.20.140

The query failed because no corresponding PTR record was found.

However, forward resolution was successful:

WIN01.benlab.local → 10.20.20.140

This demonstrated that WIN01 had a valid **A record** but did not have a corresponding **PTR record**.

Forward DNS

WIN01.benlab.local — A → 10.20.20.140

SUCCESS

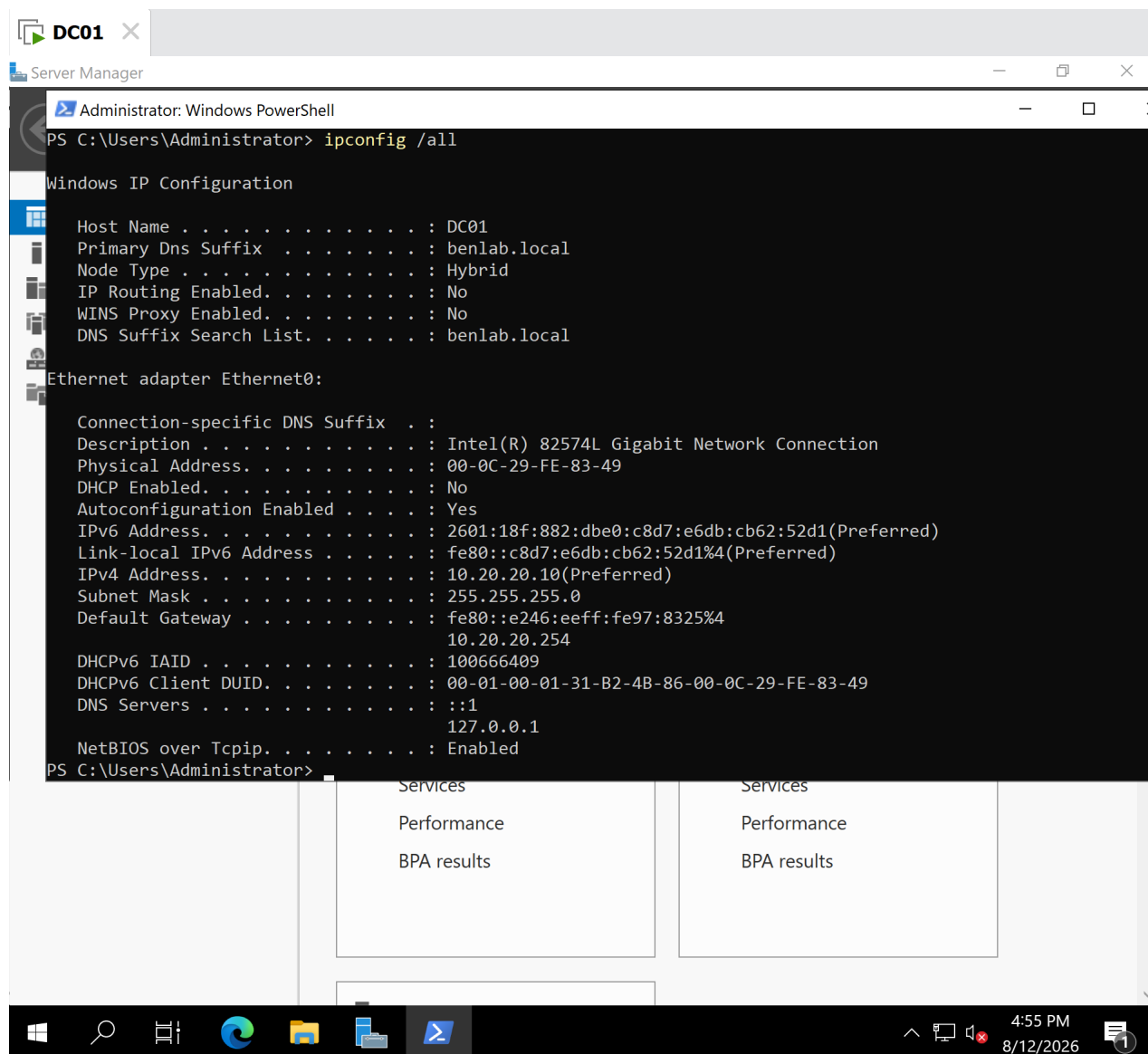
Reverse DNS

10.20.20.140 — PTR → WIN01.benlab.local

MISSING

This demonstrated that **forward and reverse DNS resolution are separate operations**.

Screenshots



The screenshot shows a Windows Server Manager window titled "DC01" with a tab for "Server Manager". Below the tab is a console window titled "Administrator: Windows PowerShell". The console displays the output of the command "ipconfig /all". The output shows the following configuration for the host DC01:

```
PS C:\Users\Administrator> ipconfig /all

Windows IP Configuration

Host Name . . . . . : DC01
Primary Dns Suffix . . . . . : benlab.local
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No
DNS Suffix Search List. . . . . : benlab.local

Ethernet adapter Ethernet0:

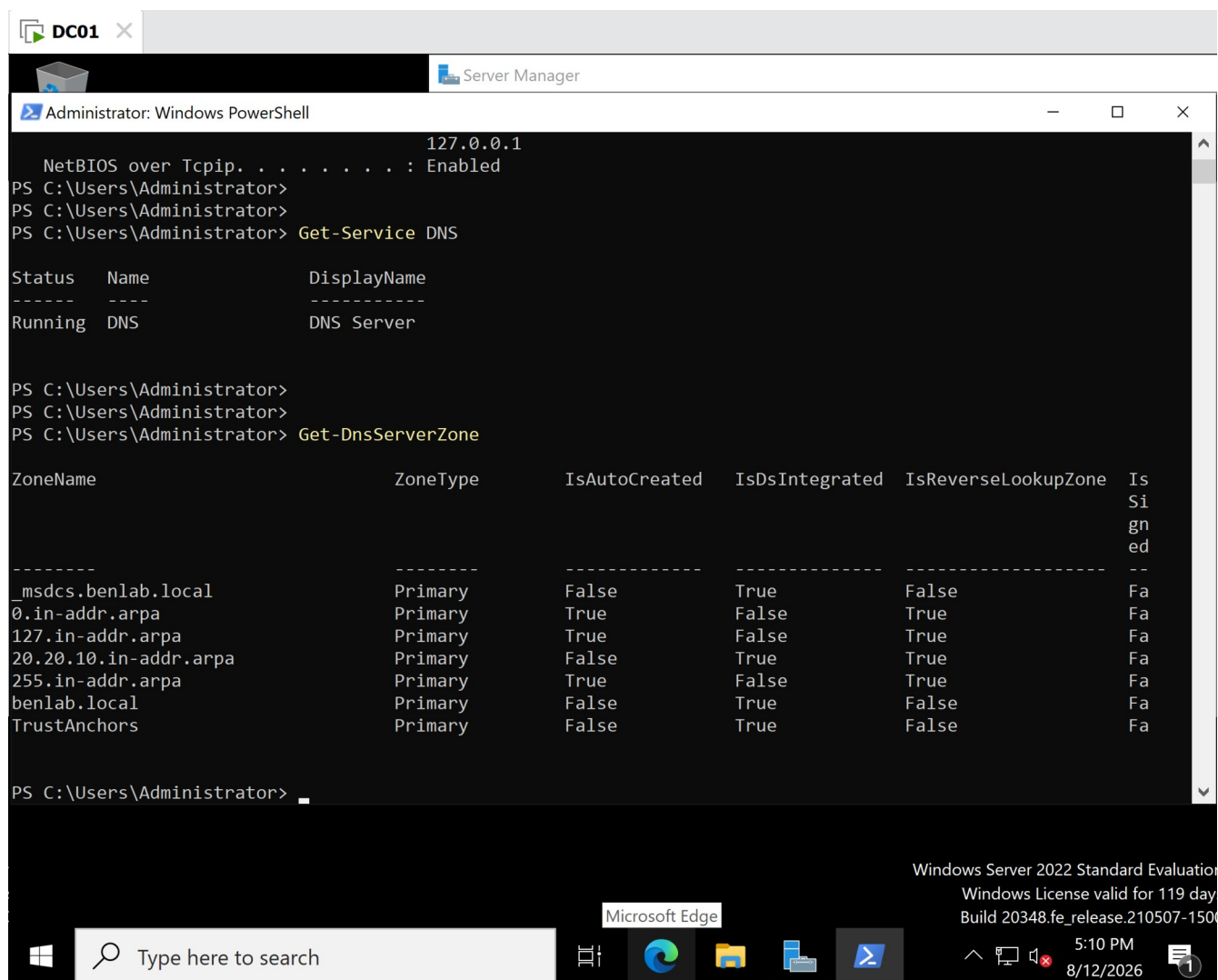
Connection-specific DNS Suffix . : 
Description . . . . . : Intel(R) 82574L Gigabit Network Connection
Physical Address. . . . . : 00-0C-29-FE-83-49
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . . : Yes
IPv6 Address. . . . . : 2601:18f:882:dbe0:c8d7:e6db:cb62:52d1(Preferred)
Link-local IPv6 Address . . . . . : fe80::c8d7:e6db:cb62:52d1%4(Preferred)
IPv4 Address. . . . . : 10.20.20.10(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : fe80::e246:eeff:fe97:8325%4
                             10.20.20.254
DHCPv6 IAID . . . . . : 100666409
DHCPv6 Client DUID. . . . . : 00-01-00-01-31-B2-4B-86-00-0C-29-FE-83-49
DNS Servers . . . . . : ::1
                             127.0.0.1
NetBIOS over Tcpip. . . . . : Enabled

PS C:\Users\Administrator>
```

The console window is overlaid on a Server Manager dashboard showing sections for "Services", "Performance", and "BPA results". The taskbar at the bottom shows the Windows logo, search icon, task view icon, and several application icons. The system clock in the bottom right corner indicates the time is 4:55 PM on 8/12/2026.

Screenshot 1 – DC01 DNS and Network Configuration

Shows the DC01 hostname, domain suffix, IPv4 address, default gateway, and configured DNS server.



Screenshot 2 – DNS Service and Zone Validation

Shows the DNS Server service running and the DNS zones hosted by DC01.

DC01

Server Manager

Administrator: Windows PowerShell

```
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-DnsServerResourceRecord -ZoneName "benlab.local"
```

HostName	RecordType	Type	Timestamp	TimeToLive	RecordData
@	A	1	8/12/2026 4:00:00 PM	00:10:00	10.20.20.10
@	NS	2	0	01:00:00	dc01.benlab.local.
@	SOA	6	0	01:00:00	[156][dc01.benlab.local...]
@	AAAA	28	8/12/2026 4:00:00 PM	00:10:00	2601:18f:882:db0:c8d7...
_msdcs	NS	2	0	01:00:00	dc01.benlab.local.
_gc._tcp.Default-First...	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][3268][dc01.be...
_kerberos._tcp.Default...	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][88][dc01.benl...
_ldap._tcp.Default-Fir...	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][389][dc01.ben...
_gc._tcp	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][3268][dc01.be...
_kerberos._tcp	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][88][dc01.benl...
_kpasswd._tcp	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][464][dc01.ben...
_ldap._tcp	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][389][dc01.ben...
_kerberos._udp	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][88][dc01.benl...
_kpasswd._udp	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][464][dc01.ben...
APP01	A	1	6/13/2026 6:00:00 PM	00:20:00	10.20.20.15
dc01	A	1	0	01:00:00	10.20.20.10
dc01	AAAA	28	0	01:00:00	2601:18f:882:db0:c8d7...
DESKTOP-0499MOA	A	1	8/11/2026 8:00:00 PM	00:20:00	10.20.20.130
DomainDnsZones	A	1	8/12/2026 4:00:00 PM	00:10:00	10.20.20.10
DomainDnsZones	AAAA	28	8/12/2026 4:00:00 PM	00:10:00	2601:18f:882:db0:c8d7...
_ldap._tcp.Default-Fir...	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][389][dc01.ben...
_ldap._tcp.DomainDnsZones	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][389][dc01.ben...
FILESRV01	A	1	8/12/2026 4:00:00 PM	00:20:00	10.20.20.50
ForestDnsZones	A	1	8/12/2026 4:00:00 PM	00:10:00	10.20.20.10
ForestDnsZones	AAAA	28	8/12/2026 4:00:00 PM	00:10:00	2601:18f:882:db0:c8d7...
_ldap._tcp.Default-Fir...	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][389][dc01.ben...
_ldap._tcp.ForestDnsZones	SRV	33	8/12/2026 4:00:00 PM	00:10:00	[0][100][389][dc01.ben...
WIN01	A	1	7/17/2026 8:00:00 PM	00:20:00	10.20.20.140

File Explorer

Build 20348.fe_release.210507-150

Type here to search

5:23 PM
8/12/2026

Screenshot 3 – DNS Resource Records

Shows records contained within the benlab.local forward lookup zone.

```
PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Resolve-DnsName 10.20.20.10

Name                                Type  TTL  Section  NameHost
----                                -
10.20.20.10.in-addr.arpa.          PTR   1200  Question  DC01.benlab.local

PS C:\Users\Administrator>
PS C:\Users\Administrator> Resolve-DnsName 10.20.20.50

Name                                Type  TTL  Section  NameHost
----                                -
10.20.20.50.in-addr.arpa          PTR   1200  Answer   FILESRV01.benlab.local

PS C:\Users\Administrator>
PS C:\Users\Administrator> Resolve-DnsName 10.20.20.140
Resolve-DnsName : 140.20.20.10.in-addr.arpa : DNS name does not exist
At line:1 char:1
+ Resolve-DnsName 10.20.20.140
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (140.20.20.10.in-addr.arpa:String) [Resolve-DnsName], Win32Exception
+ FullyQualifiedErrorId : DNS_ERROR_RCODE_NAME_ERROR,Microsoft.DnsClient.Commands.ResolveDnsName

PS C:\Users\Administrator>
```

Screenshot 4 – Reverse DNS Resolution Testing

Shows successful reverse resolution for DC01 and FILESRV01 and the failed WIN01 reverse lookup caused by the missing PTR record.

Key Learning Points

- DNS provides hostname-to-IP and IP-to-hostname resolution.
- **A records** map hostnames to IPv4 addresses.
- **PTR records** map IPv4 addresses back to hostnames.
- Forward and reverse DNS resolution are independent.
- A valid A record does not guarantee that a PTR record exists.
- `Resolve-DnsName` can test both forward and reverse resolution.
- `Get-DnsServerZone` allows administrators to inspect DNS zones through PowerShell.
- `Get-DnsServerResourceRecord` allows DNS records to be inspected from PowerShell.
- Active Directory relies heavily on DNS and SRV records for service discovery.

- Successful DNS resolution does **not** prove that the destination computer is powered on or reachable.
 - Establishing a DNS baseline before making changes is an important administration and troubleshooting practice.
-

Task 2 – Create and Verify A and PTR Records

Objective

Create and validate DNS **A and PTR records** in the `benlab.local` DNS infrastructure using DNS Manager and PowerShell.

The purpose of this task was to practice both **forward and reverse DNS record administration** and understand the relationship between A and PTR records.

Test DNS Record

A test hostname was created specifically for DNS administration practice.

The following information was used:

- **Hostname:** TESTSERVER01
- **FQDN:** TESTSERVER01.benlab.local
- **IPv4 Address:** 10.20.20.200
- **Forward Lookup Zone:** benlab.local
- **Reverse Lookup Zone:** 20.20.10.in-addr.arpa

TESTSERVER01 was used as a DNS test record and did not require an actual server to be running at the assigned IP address.

Create TESTSERVER01 A Record

DNS Manager was opened on DC01 and the following location was selected:

DC01

└─ Forward Lookup Zones

└─ benlab.local

The **New Host (A or AAAA)** option was used to create the record.

The following values were entered:

Name: TESTSERVER01
FQDN: TESTSERVER01.benlab.local
IP Address: 10.20.20.200

The host record was successfully created in the `benlab.local` forward lookup zone.

The resulting A record provides:

TESTSERVER01.benlab.local → 10.20.20.200

Create Associated PTR Record

While creating the TESTSERVER01 A record, the following option was enabled:

Create associated pointer (PTR) record

This instructed DNS Manager to create the corresponding PTR record in the existing reverse lookup zone.

The PTR record provides the opposite mapping:

10.20.20.200 → TESTSERVER01.benlab.local

This demonstrated that an **A record** and a **PTR record** perform complementary but separate DNS functions.

Forward Lookup Verification

The new A record was verified from PowerShell using:

```
Resolve-DnsName TESTSERVER01.benlab.local
```

The query successfully returned:

Name: TESTSERVER01.benlab.local
Type: A
IPAddress: 10.20.20.200

This confirmed successful **forward DNS resolution**:

Hostname — A Record → IPv4 Address

TESTSERVER01.benlab.local

|



10.20.20.200

Reverse Lookup Verification

Reverse DNS resolution was tested using:

Resolve-DnsName 10.20.20.200

The query successfully returned:

Type: PTR

NameHost: TESTSERVER01.benlab.local

This confirmed successful **reverse DNS resolution**:

IPv4 Address — PTR Record → Hostname

10.20.20.200

|



TESTSERVER01.benlab.local

Both the A and PTR records were therefore successfully created and verified.

DNS Resolution vs. Host Availability

TESTSERVER01 was only created as a DNS record for testing purposes. No physical or virtual server was actually operating at 10.20.20.200.

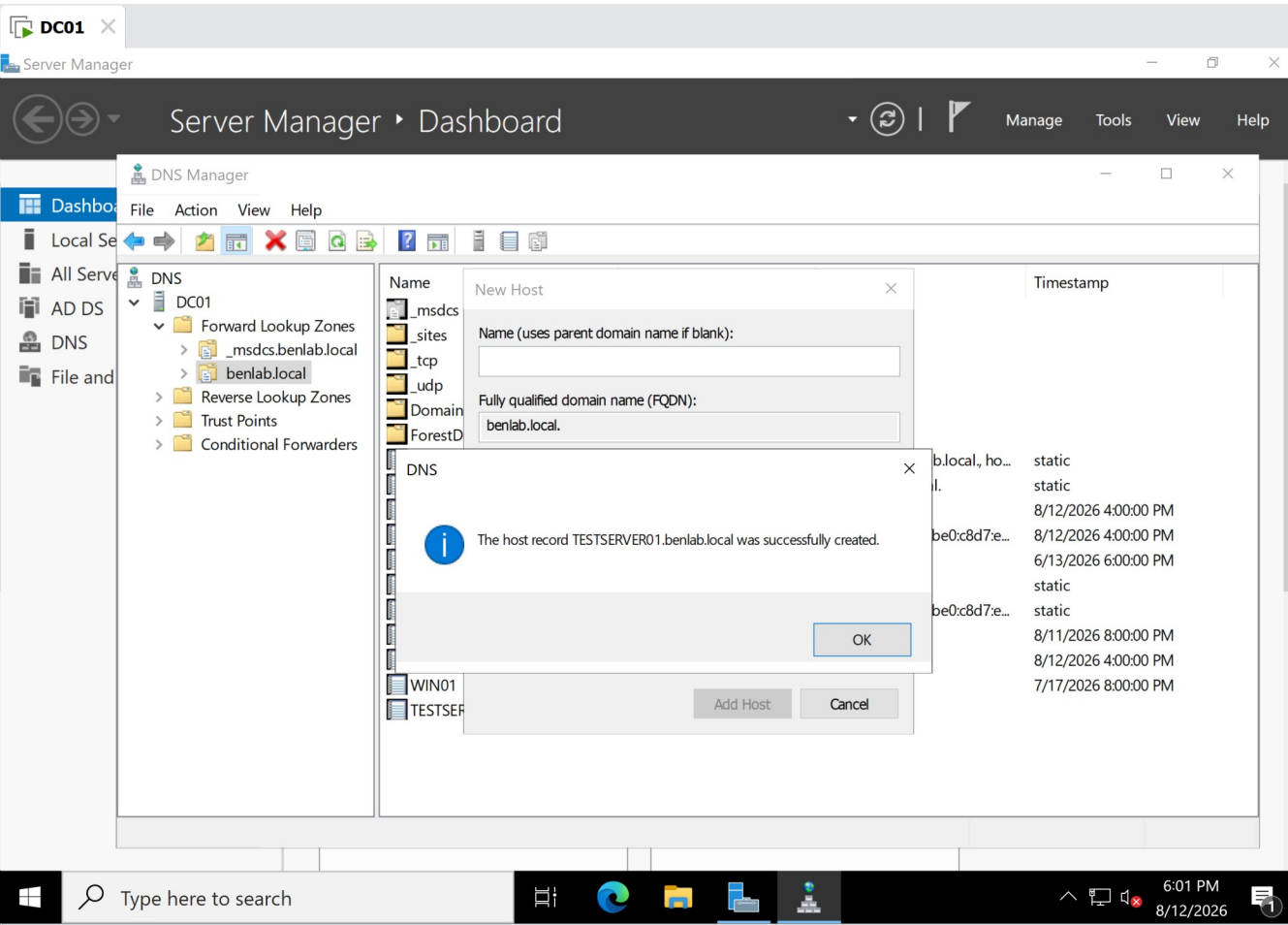
Despite this, both DNS queries succeeded.

This demonstrated an important troubleshooting concept:

Successful DNS resolution does not necessarily mean that the destination computer is online or reachable.

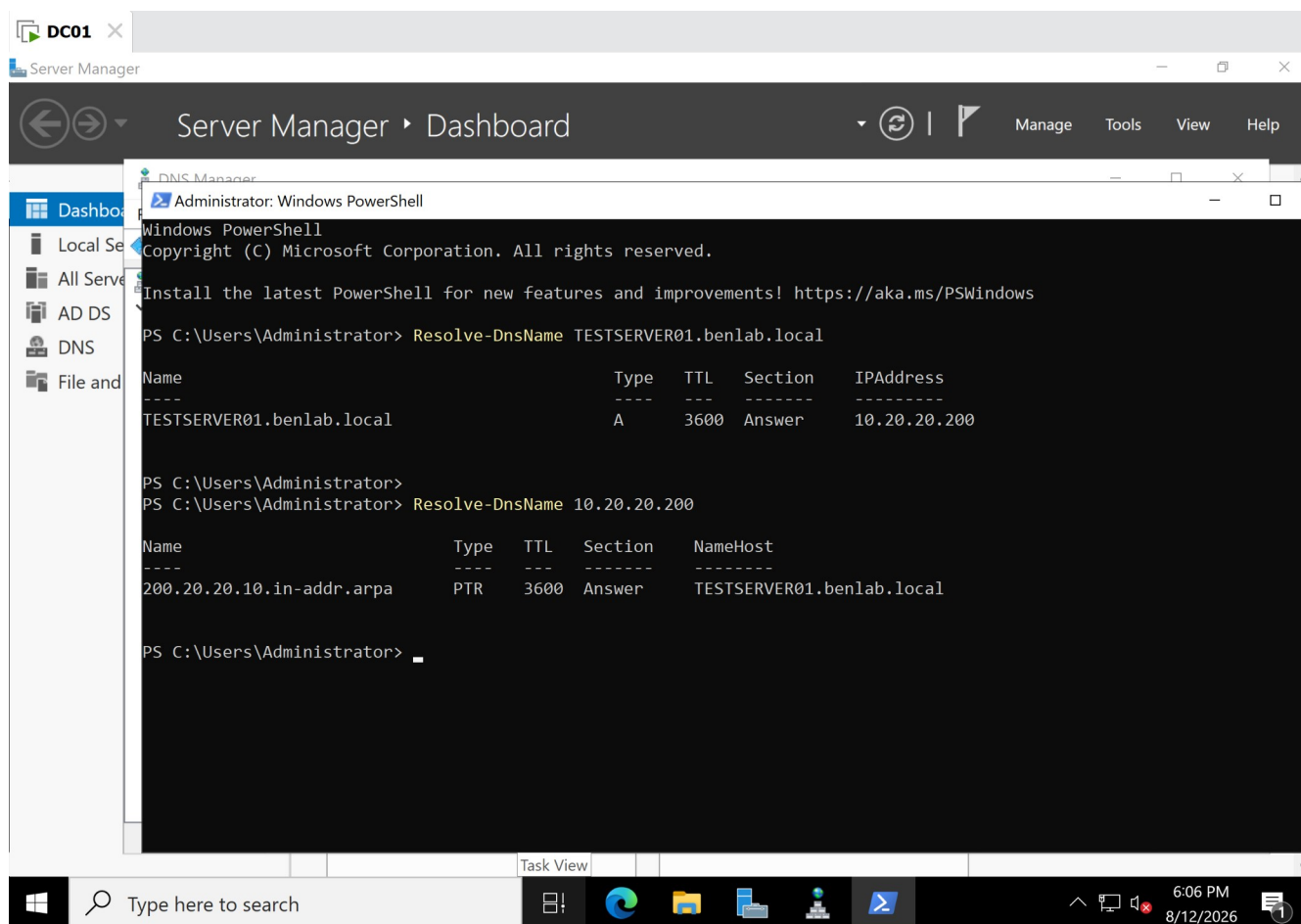
DNS confirms that a record exists associating a hostname and IP address. Additional network testing would be required to determine whether an actual host is available at that address.

Screenshots



Screenshot 1 – TESTSERVER01 Host Record Creation

Shows confirmation that the TESTSERVER01.benlab.local host record was successfully created.



Screenshot 2 – Forward and Reverse DNS Verification

Shows successful PowerShell resolution of:

TESTSERVER01.benlab.local → 10.20.20.200

and:

10.20.20.200 → TESTSERVER01.benlab.local

This screenshot provides evidence that both the A and associated PTR records were functioning correctly.

Key Learning Points

- An **A record** maps a hostname to an IPv4 address.
- A **PTR record** maps an IPv4 address back to a hostname.
- DNS Manager can automatically create a corresponding PTR record when creating an A record.
- Forward and reverse DNS resolution should be tested independently.
- `Resolve-DnsName` can verify both A and PTR records.
- DNS records can exist even when no actual device is operating at the configured IP address.

- Successful name resolution does not prove network connectivity or host availability.
 - Creating both A and PTR records provides more complete DNS name-resolution coverage.
 - DNS changes should be verified after configuration rather than assuming successful creation means resolution is functioning.
-

Task 3 – Troubleshoot and Repair DNS Record Inconsistency

Objective

Troubleshoot a DNS resolution inconsistency involving the TESTSERVER01 host record.

The goal was to identify why forward name resolution worked while reverse name resolution failed, inspect the DNS records, correct the inconsistency, and verify successful resolution in both directions.

Initial DNS Configuration

TESTSERVER01 had the following DNS A record:

Hostname: TESTSERVER01

FQDN: TESTSERVER01.benlab.local

IP Address: 10.20.20.201

Forward resolution was tested using PowerShell:

```
Resolve-DnsName TESTSERVER01.benlab.local
```

The query successfully returned:

```
TESTSERVER01.benlab.local      A      10.20.20.201
```

This confirmed that the **A record was functioning correctly**.

Reverse Lookup Failure

Reverse DNS resolution was then tested:

```
Resolve-DnsName 10.20.20.201
```

The query returned:

DNS name does not exist

This demonstrated that although the hostname could be translated to its IP address, DNS could not translate the IP address back to the hostname.

The problem therefore involved **reverse DNS resolution rather than forward DNS resolution**.

DNS Manager Investigation

DNS Manager was opened on DC01, and the following reverse lookup zone was inspected:

20.20.10.in-addr.arpa

Existing PTR records were present for several systems in the 10.20.20.0/24 network, but there was no PTR record for:

10.20.20.201

This identified the root cause of the failed reverse lookup.

Finding: The A record for TESTSERVER01 existed, but its corresponding **PTR record was missing**.

PTR Record Repair

A new PTR record was created in the reverse lookup zone.

The completed mapping was:

10.20.20.201 → TESTSERVER01.benlab.local

DNS Manager was then refreshed to verify that the new PTR record appeared successfully in the reverse lookup zone.

Forward and Reverse Resolution Verification

After repairing the PTR record, forward resolution was tested again:

Resolve-DnsName TESTSERVER01.benlab.local

Result:

TESTSERVER01.benlab.local → 10.20.20.201

Reverse resolution was then tested:

Resolve-DnsName 10.20.20.201

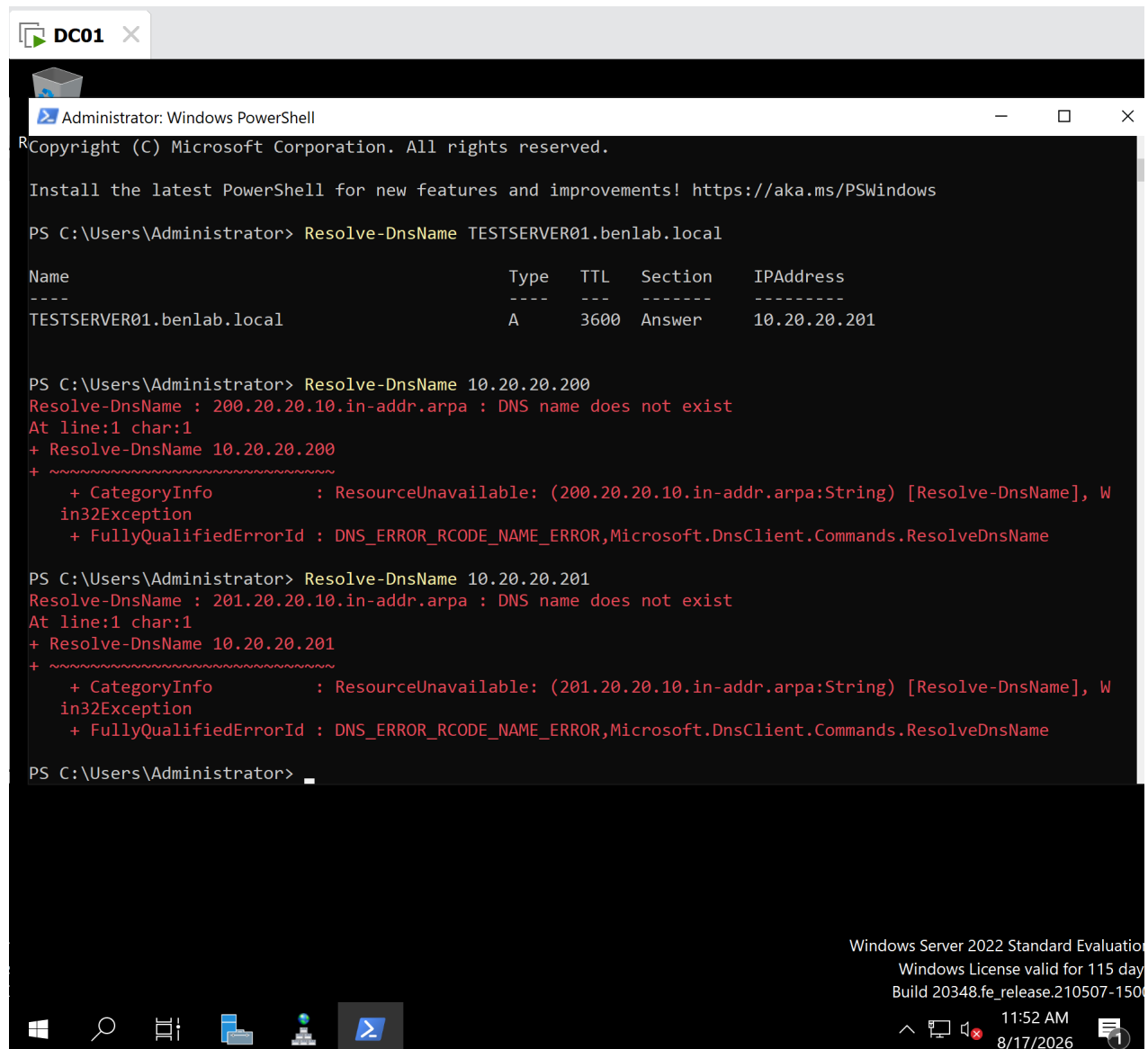
Result:

10.20.20.201 → TESTSERVER01.benlab.local

Both queries completed successfully.

This confirmed that the **A and PTR records were now consistent**.

Screenshots



```
DC01 x
Administrator: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Administrator> Resolve-DnsName TESTSERVER01.benlab.local

Name                                     Type  TTL  Section  IPAddress
----
TESTSERVER01.benlab.local                A      3600  Answer   10.20.20.201

PS C:\Users\Administrator> Resolve-DnsName 10.20.20.200
Resolve-DnsName : 200.20.20.10.in-addr.arpa : DNS name does not exist
At line:1 char:1
+ Resolve-DnsName 10.20.20.200
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (200.20.20.10.in-addr.arpa:String) [Resolve-DnsName], W
in32Exception
+ FullyQualifiedErrorId : DNS_ERROR_RCODE_NAME_ERROR,Microsoft.DnsClient.Commands.ResolveDnsName

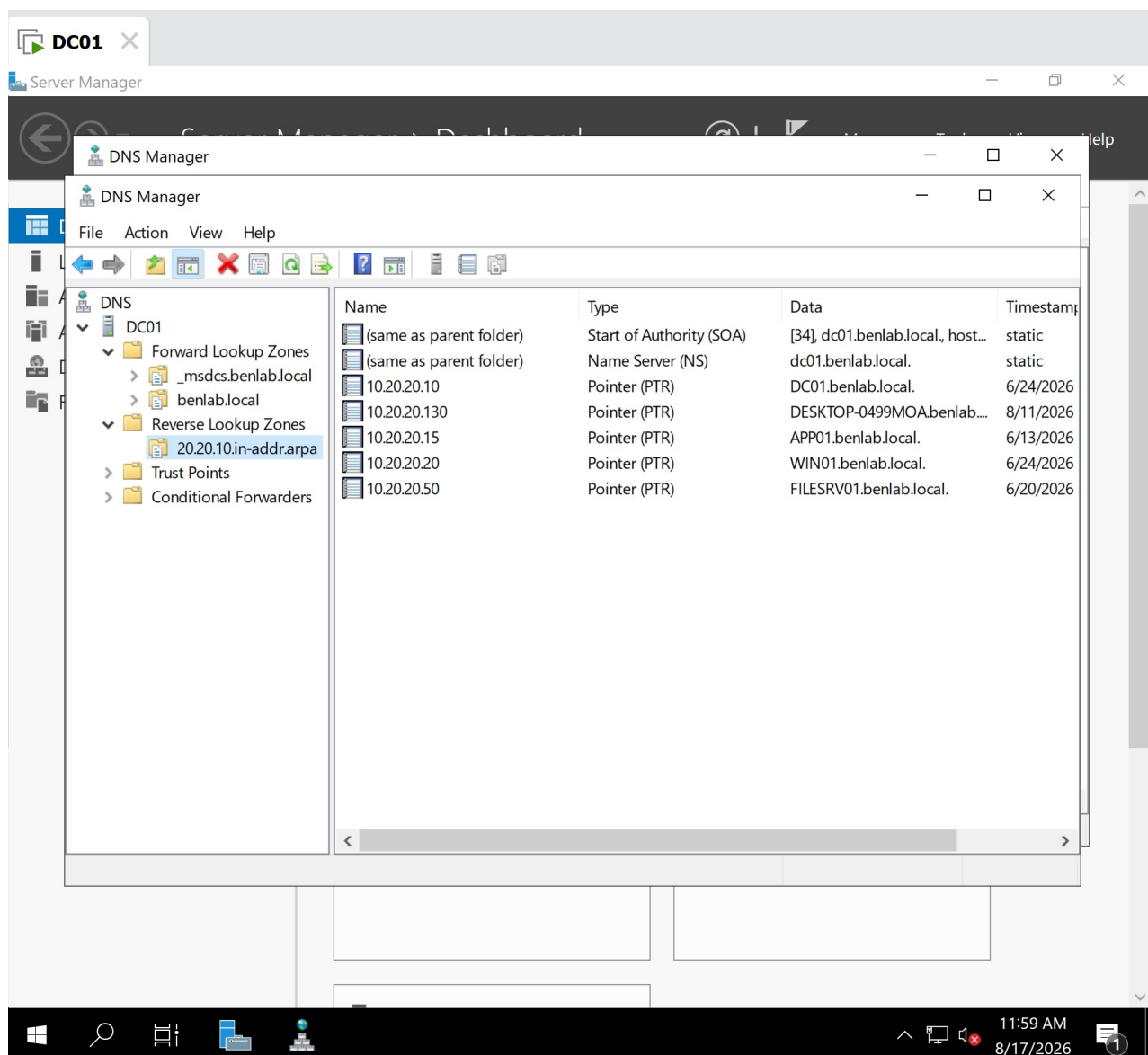
PS C:\Users\Administrator> Resolve-DnsName 10.20.20.201
Resolve-DnsName : 201.20.20.10.in-addr.arpa : DNS name does not exist
At line:1 char:1
+ Resolve-DnsName 10.20.20.201
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (201.20.20.10.in-addr.arpa:String) [Resolve-DnsName], W
in32Exception
+ FullyQualifiedErrorId : DNS_ERROR_RCODE_NAME_ERROR,Microsoft.DnsClient.Commands.ResolveDnsName

PS C:\Users\Administrator> _

Windows Server 2022 Standard Evaluation
Windows License valid for 115 day
Build 20348.fe_release.210507-1500
11:52 AM
8/17/2026
```

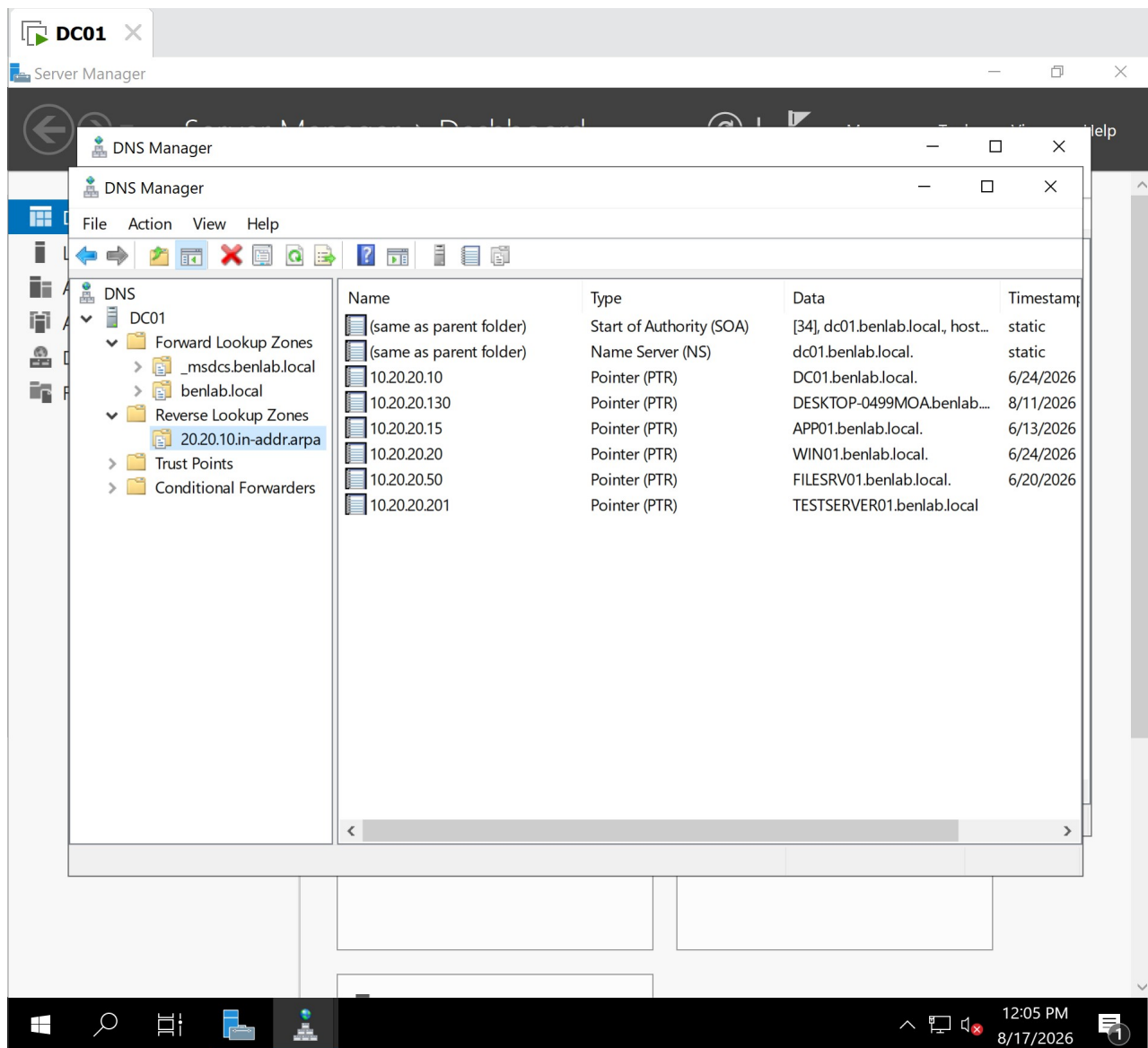
Screenshot 1 – Reverse DNS Failure

PowerShell showing that the forward lookup for `TESTSERVER01.benlab.local` succeeded while the reverse lookup failed because the PTR record was missing.



Screenshot 2 – Reverse Lookup Zone Investigation

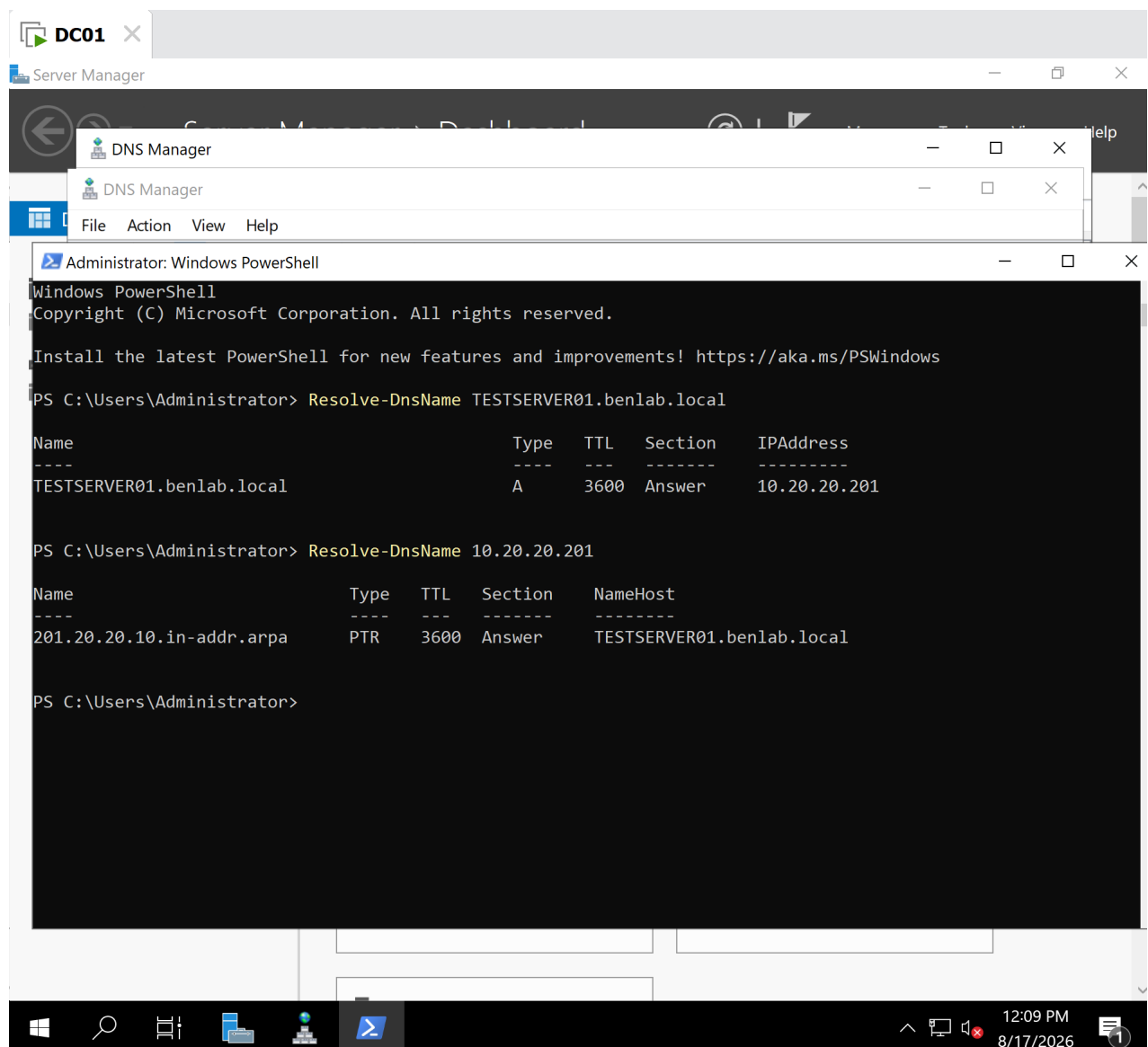
DNS Manager showing the 20.20.10.in-addr.arpa reverse lookup zone without a PTR record for 10.20.20.201.



Screenshot 3 – PTR Record Created

DNS Manager showing the newly created PTR record:

10.20.20.201 → TESTSERVER01.benlab.local



Screenshot 4 – Final DNS Verification

PowerShell showing successful forward and reverse resolution:

TESTSERVER01.benlab.local → 10.20.20.201

10.20.20.201 → TESTSERVER01.benlab.local

Key Learning Points

- An **A record** provides forward DNS resolution from a hostname to an IPv4 address.
- A **PTR record** provides reverse DNS resolution from an IP address to a hostname.
- Successful forward resolution does **not** guarantee successful reverse resolution.
- A missing PTR record does not necessarily mean the entire DNS service is broken.
- `Resolve-DnsName` can be used to test both forward and reverse DNS resolution.

- DNS Manager can be used to inspect and manually repair DNS resource records.
 - Comparing forward and reverse lookup results is an effective DNS troubleshooting technique.
 - After repairing DNS records, administrators should **verify the solution rather than assuming the configuration change worked**.
-

Result

Status: Completed Successfully

TESTSERVER01 now has consistent forward and reverse DNS resolution:

A record: TESTSERVER01.benlab.local → 10.20.20.201

PTR record: 10.20.20.201 → TESTSERVER01.benlab.local

This task demonstrated a realistic DNS troubleshooting workflow: **identify the symptom** → **isolate forward vs. reverse resolution** → **inspect DNS records** → **identify the missing record** → **repair it** → **verify the result**.

Task 4 – Forward Lookup Zone Administration and CNAME Records

Objective

The objective of this task was to perform additional administration within the benlab.local Forward Lookup Zone and understand how DNS aliases (CNAME records) work.

The task demonstrated the relationship between A records and CNAME records, how an alias continues functioning when the IP address of its target host changes, and the important distinction between successful DNS name resolution and actual network connectivity.

Environment

The task was performed on:

- **DNS Server:** DC01
- **DNS Server IP:** 10.20.20.10
- **Domain:** benlab.local

- **Forward Lookup Zone:** benlab.local
- **Test host:** TESTSERVER01.benlab.local
- **Initial test IP:** 10.20.20.201
- **Updated test IP:** 10.20.20.202
- **DNS Alias:** intranet.benlab.local

TESTSERVER01 was used as a DNS test record and does not represent an active server at the configured test IP address.

CNAME Record Creation

DNS Manager was opened on DC01 and the following location was selected:

DC01

└─ Forward Lookup Zones
 └─ benlab.local

A new **Alias (CNAME)** resource record was created with the following configuration:

Alias name:

intranet

Fully Qualified Domain Name:

intranet.benlab.local

Target host:

TESTSERVER01.benlab.local

The resulting relationship was:

intranet.benlab.local

↓

CNAME

↓

TESTSERVER01.benlab.local

Unlike an A record, the CNAME does not directly contain the IPv4 address. Instead, it references another hostname.

CNAME Resolution Verification

PowerShell was used to verify the new alias:

```
Resolve-DnsName intranet.benlab.local
```

DNS returned the CNAME target:

```
intranet.benlab.local
```

↓

```
TESTSERVER01.benlab.local
```

DNS then resolved the A record associated with TESTSERVER01:

```
TESTSERVER01.benlab.local
```

↓

```
10.20.20.201
```

This confirmed the complete resolution process:

```
intranet.benlab.local
```

↓ CNAME

```
TESTSERVER01.benlab.local
```

↓ A

```
10.20.20.201
```

Testing a Change to the Target A Record

To demonstrate an important administrative benefit of CNAME records, the IP address associated with TESTSERVER01 was changed.

The A record was modified from:

```
10.20.20.201
```

to:

```
10.20.20.202
```

The intranet CNAME record itself was **not modified**.

The DNS client cache was then cleared:

```
Clear-DnsClientCache
```

The updated A record was verified:

```
Resolve-DnsName TESTSERVER01.benlab.local
```

The result showed:

TESTSERVER01.benlab.local → 10.20.20.202

CNAME Verification After the IP Change

The alias was queried again:

```
Resolve-DnsName intranet.benlab.local
```

The result showed:

```
intranet.benlab.local
```

```
    ↓ CNAME
```

```
TESTSERVER01.benlab.local
```

```
    ↓ A
```

```
10.20.20.202
```

The alias continued to function even though the underlying server IP address had changed.

This demonstrated an important advantage of CNAME records: a friendly or service-oriented DNS name can remain unchanged while administrators modify the host information behind it.

DNS Resolution vs. Network Connectivity

A final troubleshooting test was performed to distinguish DNS resolution from actual network connectivity.

First:

```
Resolve-DnsName TESTSERVER01.benlab.local
```

successfully returned:

```
10.20.20.202
```

This confirmed that the **DNS Server running on DC01 was functioning correctly** and could resolve the hostname.

Connectivity was then tested:

```
ping TESTSERVER01.benlab.local
```

Windows successfully resolved the hostname:

```
Pinging TESTSERVER01.benlab.local [10.20.20.202]
```

However, the result returned:

```
Reply from 10.20.20.10: Destination host unreachable.
```

This occurred because 10.20.20.202 was created as a DNS test record and no actual TESTSERVER01 device was operating at that IP address.

Therefore:

DNS Resolution → SUCCESS

Network Connectivity → FAILED

This demonstrated that successful DNS resolution does not guarantee that the destination system is online or reachable.

Troubleshooting Logic

The task established the following troubleshooting approach:

Can the hostname be resolved?

↓

YES

↓

DNS is resolving the name

↓

Can the destination be reached?

↓

NO

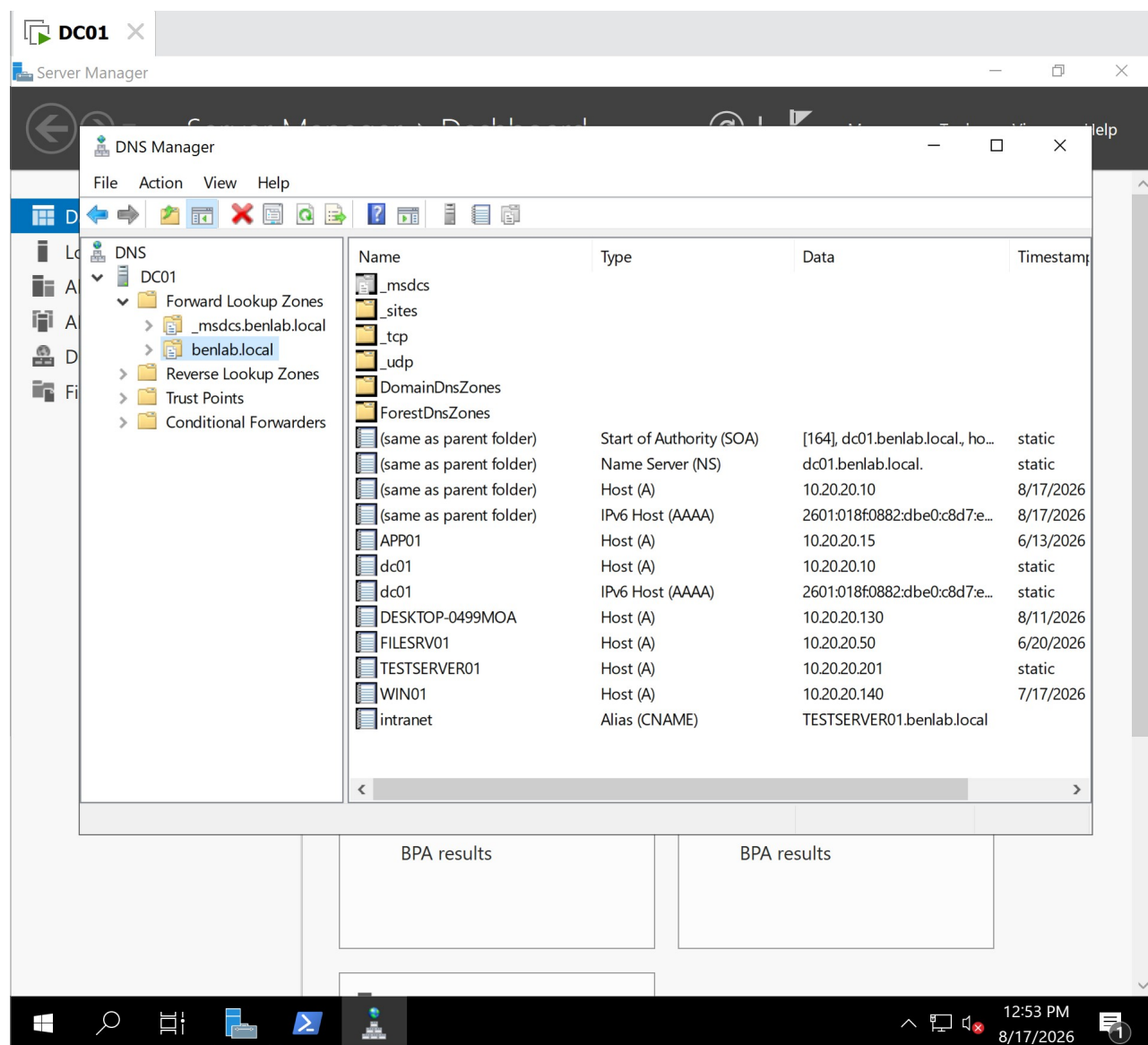
↓

Investigate:

- Host availability
- Network connectivity
- Routing
- Firewall
- Required service

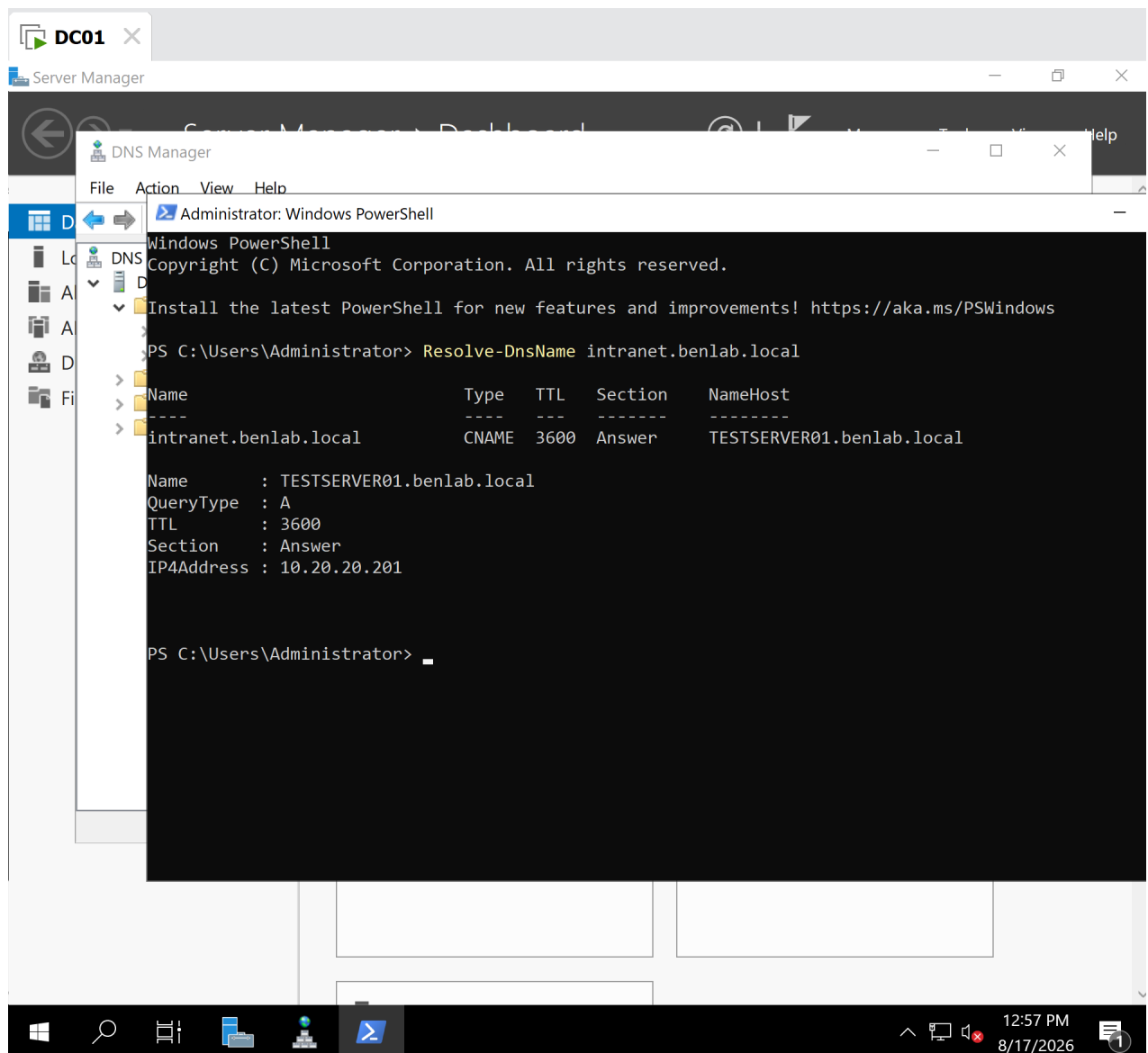
A connectivity failure should therefore not automatically be diagnosed as a DNS problem.

Screenshots



Screenshot 1 – CNAME Record in DNS Manager

Shows the intranet CNAME record configured in the benlab.local Forward Lookup Zone and pointing to:
TESTSERVER01.benlab.local



Screenshot 2 – Initial CNAME Resolution

Shows:

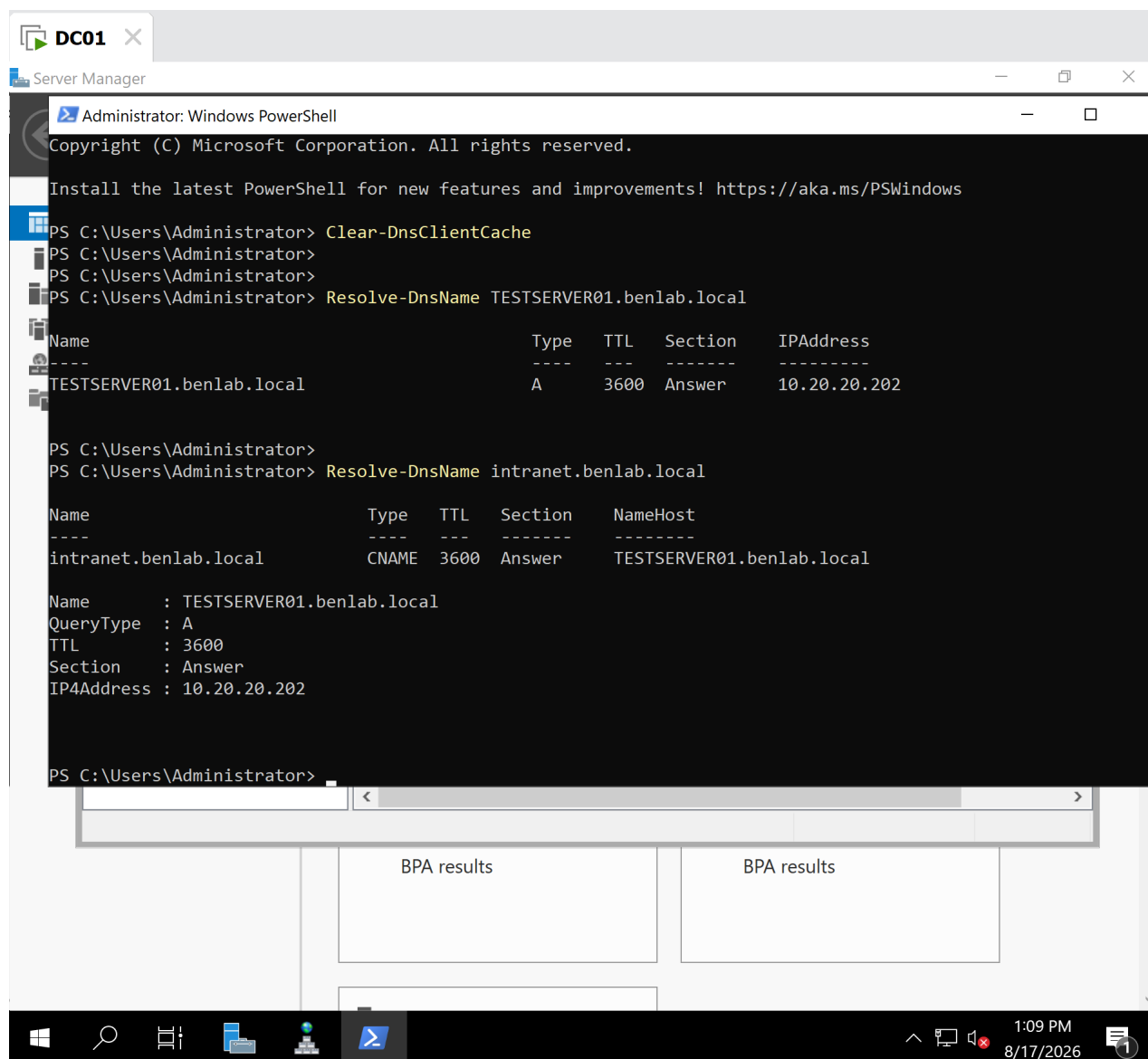
`Resolve-DnsName intranet.benlab.local`

resolving:

`intranet.benlab.local`

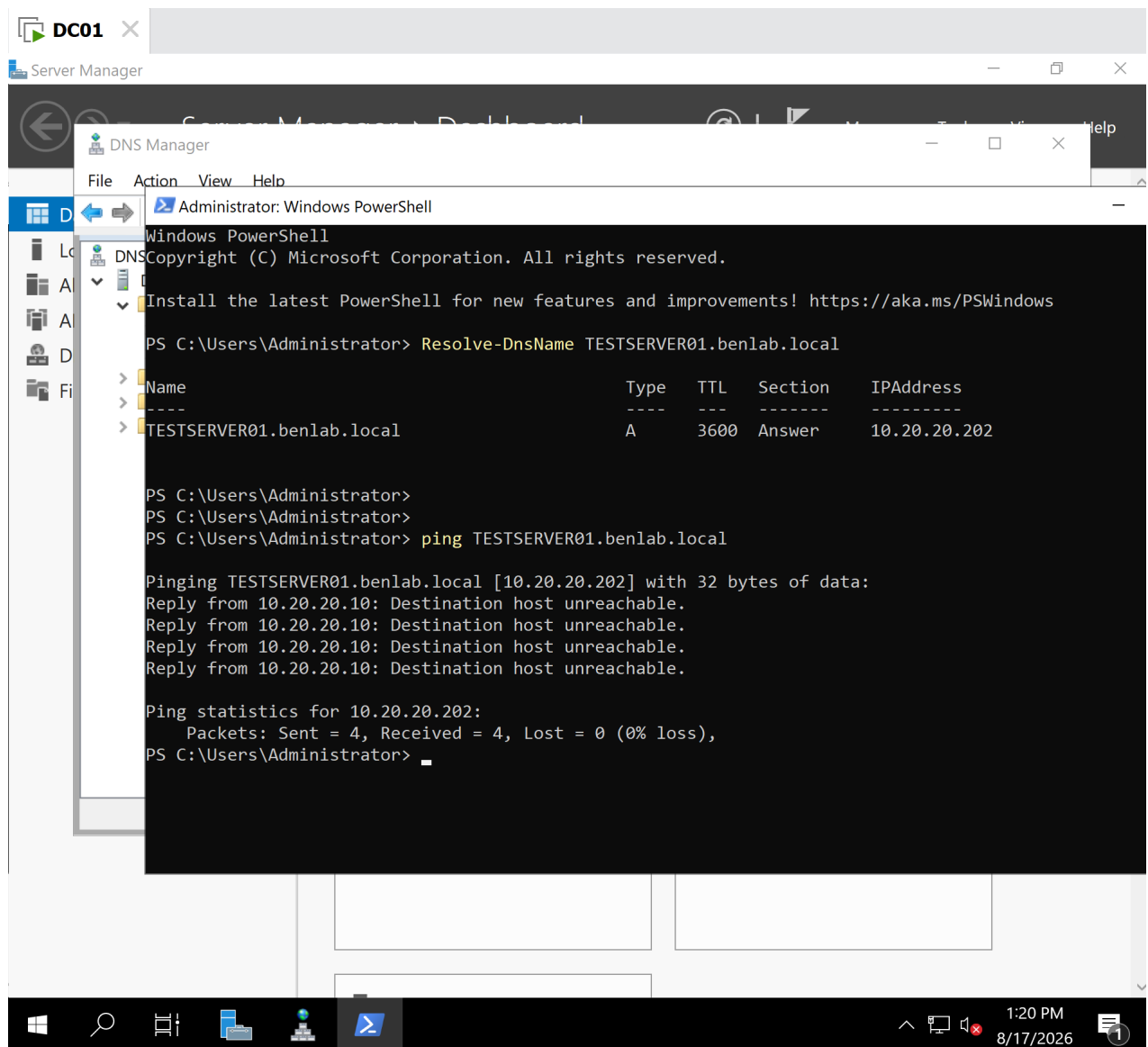
→ `TESTSERVER01.benlab.local`

→ `10.20.20.201`



Screenshot 3 – CNAME Resolution After A Record Change

Shows the A record changed to 10.20.20.202 and demonstrates that the unchanged intranet CNAME automatically resolves through the updated target.



Screenshot 4 – DNS Resolution vs. Connectivity

Shows successful:

`Resolve-DnsName TESTSERVER01.benlab.local`

returning `10.20.20.202`, followed by:

`ping TESTSERVER01.benlab.local`

successfully resolving the hostname but returning `Destination host unreachable`.

This provides evidence that **DNS resolution and network connectivity are separate mechanisms**.

Key Learning Points

- A **Forward Lookup Zone** translates hostnames into IP addresses.
- An **A record** maps a hostname directly to an IPv4 address.

- A **CNAME record** maps one hostname to another hostname.
 - `intranet.benlab.local` can act as a friendly alias for `TESTSERVER01.benlab.local`.
 - Changing the target host's A record does not require changing the CNAME.
 - `Clear-DnsClientCache` removes locally cached DNS responses before retesting.
 - `Resolve-DnsName` is useful for directly testing DNS name resolution.
 - Successful DNS resolution does **not** prove that the destination host is online or reachable.
 - `ping` may successfully resolve a hostname while communication with the resulting IP address still fails.
 - DNS troubleshooting should separate **name-resolution problems** from **network-connectivity problems**.
 - In this lab, **DC01 at 10.20.20.10 is the actual DNS server**; 10.20.20.202 is only the test IP stored in the TESTSERVER01 A record.
-

Result

The `benlab.local` Forward Lookup Zone was administered by creating and testing a CNAME alias, modifying the target A record, verifying automatic alias resolution to the updated address, and demonstrating the difference between DNS resolution and network connectivity.

Task 5 – DNS Troubleshooting

Objective

Troubleshoot a DNS name-resolution failure on a domain workstation, identify the root cause, restore the correct DNS client configuration, and verify successful internal DNS resolution.

Environment

- **Domain:** `benlab.local`
- **DNS Server / Domain Controller:** DC01
- **DC01 IP Address:** 10.20.20.10

- **Client Workstation:** CLIENT02
 - **Test DNS Record:** TESTSERVER01.benlab.local
 - **TESTSERVER01 IP:** 10.20.20.202
 - **Incorrect DNS Server used for troubleshooting:** 8.8.8.8
-

Scenario

A DNS troubleshooting scenario was simulated by changing CLIENT02's configured DNS server from the internal DNS server, DC01 (10.20.20.10), to the external Google DNS server (8.8.8.8).

After the change, CLIENT02 maintained IP connectivity to DC01 but could no longer resolve resources within the internal benlab.local DNS namespace.

This simulated a common enterprise issue where network connectivity remains functional while DNS name resolution fails.

Troubleshooting Procedure

Step 1 – Verify IP Connectivity

Connectivity from CLIENT02 to DC01 was tested using:

```
ping 10.20.20.10
```

The ping succeeded with **0% packet loss**, confirming that CLIENT02 could communicate with DC01 at the IP layer.

This established that the problem was not basic network connectivity.

Step 2 – Test Internal DNS Resolution

Internal DNS resolution was tested using:

```
Resolve-DnsName DC01.benlab.local
```

```
Resolve-DnsName TESTSERVER01.benlab.local
```

Both DNS queries failed with:

```
DNS name does not exist
```

Because IP connectivity was successful while hostname resolution failed, the investigation was narrowed to DNS.

Step 3 – Inspect DNS Client Configuration

The DNS server configured on CLIENT02 was examined using:

```
Get-DnsClientServerAddress -AddressFamily IPv4
```

The result showed:

```
Ethernet0    IPv4    {8.8.8.8}
```

This identified the source of the problem.

CLIENT02 was querying Google Public DNS instead of the internal DNS server on DC01.

Root Cause

CLIENT02 was incorrectly configured to use:

```
8.8.8.8
```

instead of the internal DNS server:

```
10.20.20.10
```

Google Public DNS can resolve public Internet domains but does not contain the private Active Directory DNS records maintained by DC01 for the `benlab.local` namespace.

As a result, CLIENT02 retained normal IP connectivity but lost the ability to resolve internal domain resources by hostname.

Resolution

The DNS client configuration on CLIENT02 was restored to DC01 using Administrator PowerShell:

```
Set-DnsClientServerAddress -InterfaceAlias "Ethernet0" -ServerAddresses 10.20.20.10
```

The DNS client cache was then cleared:

```
Clear-DnsClientCache
```

The corrected DNS configuration was verified:

```
Get-DnsClientServerAddress -AddressFamily IPv4
```

The result showed:

```
Ethernet0    IPv4    {10.20.20.10}
```

CLIENT02 was therefore restored to the correct internal DNS infrastructure.

Final Verification

Internal DNS resolution was tested again:

```
Resolve-DnsName DC01.benlab.local
```

The query successfully returned:

DC01.benlab.local A 10.20.20.10

The test server record was then queried:

Resolve-DnsName TESTSERVER01.benlab.local

The query successfully returned:

TESTSERVER01.benlab.local A 10.20.20.202

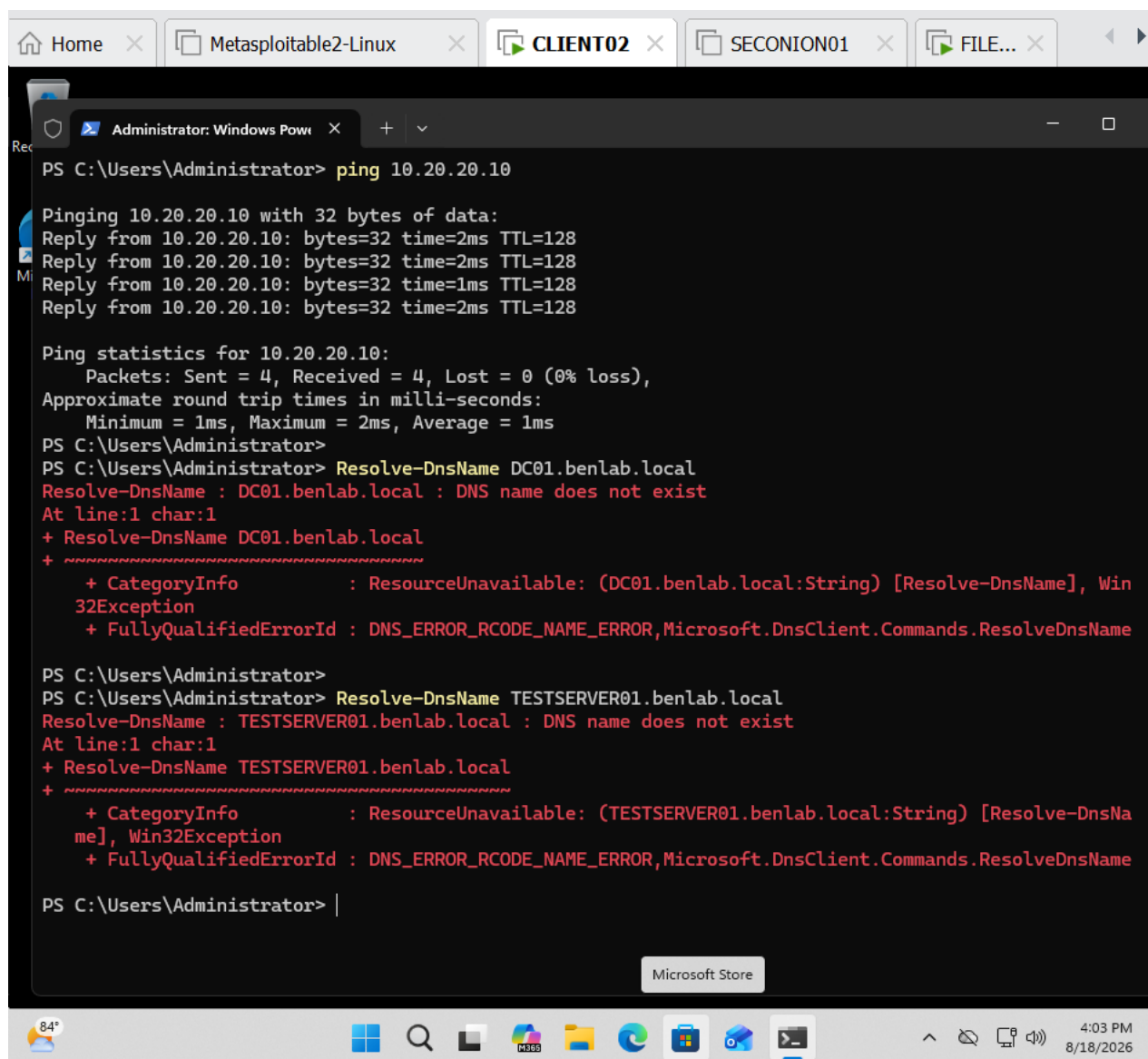
Both internal DNS records resolved successfully, confirming that DNS functionality had been restored.

Result

Task completed successfully.

The DNS failure was isolated from basic network connectivity. The incorrect DNS client configuration was identified as the root cause, CLIENT02 was restored to the internal DNS server, and successful internal name resolution was verified.

Screenshots / Evidence



```
PS C:\Users\Administrator> ping 10.20.20.10

Pinging 10.20.20.10 with 32 bytes of data:
Reply from 10.20.20.10: bytes=32 time=2ms TTL=128
Reply from 10.20.20.10: bytes=32 time=2ms TTL=128
Reply from 10.20.20.10: bytes=32 time=1ms TTL=128
Reply from 10.20.20.10: bytes=32 time=2ms TTL=128

Ping statistics for 10.20.20.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 2ms, Average = 1ms
PS C:\Users\Administrator>
PS C:\Users\Administrator> Resolve-DnsName DC01.benlab.local
Resolve-DnsName : DC01.benlab.local : DNS name does not exist
At line:1 char:1
+ Resolve-DnsName DC01.benlab.local
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (DC01.benlab.local:String) [Resolve-DnsName], Win
32Exception
+ FullyQualifiedErrorId : DNS_ERROR_RCODE_NAME_ERROR,Microsoft.DnsClient.Commands.ResolveDnsName

PS C:\Users\Administrator>
PS C:\Users\Administrator> Resolve-DnsName TESTSERVER01.benlab.local
Resolve-DnsName : TESTSERVER01.benlab.local : DNS name does not exist
At line:1 char:1
+ Resolve-DnsName TESTSERVER01.benlab.local
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (TESTSERVER01.benlab.local:String) [Resolve-DnsNa
me], Win32Exception
+ FullyQualifiedErrorId : DNS_ERROR_RCODE_NAME_ERROR,Microsoft.DnsClient.Commands.ResolveDnsName

PS C:\Users\Administrator> |
```

Screenshot 1 – DNS Resolution Failure

Shows CLIENT02 successfully pinging DC01 at 10.20.20.10 while DNS queries for DC01.benlab.local and TESTSERVER01.benlab.local fail.

Purpose: Demonstrates that IP connectivity was functional while DNS name resolution was failing.

The screenshot shows a Windows PowerShell terminal window with the following content:

```
PS C:\Users\Administrator> Resolve-DnsName DC01.benlab.local
Resolve-DnsName : DC01.benlab.local : DNS name does not exist
At line:1 char:1
+ Resolve-DnsName DC01.benlab.local
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (DC01.benlab.local:String) [Resolve-DnsName], Win32Exception
+ FullyQualifiedErrorId : DNS_ERROR_RCODE_NAME_ERROR,Microsoft.DnsClient.Commands.ResolveDnsName

PS C:\Users\Administrator>
PS C:\Users\Administrator> Resolve-DnsName TESTSERVER01.benlab.local
Resolve-DnsName : TESTSERVER01.benlab.local : DNS name does not exist
At line:1 char:1
+ Resolve-DnsName TESTSERVER01.benlab.local
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (TESTSERVER01.benlab.local:String) [Resolve-DnsName], Win32Exception
+ FullyQualifiedErrorId : DNS_ERROR_RCODE_NAME_ERROR,Microsoft.DnsClient.Commands.ResolveDnsName

PS C:\Users\Administrator> Get-DnsClientServerAddress -AddressFamily IPv4

InterfaceAlias          Interface Index Address ServerAddresses
-----
Ethernet0               14 IPv4 {8.8.8.8}
Loopback Pseudo-Interface 1 1 IPv4 {}
```

At the bottom of the terminal window, there is a Microsoft 365 Copilot button and a taskbar showing the system clock as 4:07 PM on 8/18/2026.

Screenshot 2 – Incorrect DNS Server Identified

Shows:

Get-DnsClientServerAddress -AddressFamily IPv4

returning:

Ethernet0 IPv4 {8.8.8.8}

Purpose: Documents the incorrect DNS client configuration and identifies the root cause of the name-resolution failure.

```
PS C:\Users\Administrator> Get-DnsClientServerAddress -AddressFamily IPv4

InterfaceAlias      Interface Address ServerAddresses
-----
Ethernet0           14 IPv4    {8.8.8.8}
Loopback Pseudo-Interface 1  1 IPv4    {}

PS C:\Users\Administrator> Set-DnsClientServerAddress -InterfaceAlias "Ethernet0" -ServerAddresses 10.20.20.10
PS C:\Users\Administrator> Clear-DnsClientCache
PS C:\Users\Administrator> Get-DnsClientServerAddress -AddressFamily IPv4

InterfaceAlias      Interface Address ServerAddresses
-----
Ethernet0           14 IPv4    {10.20.20.10}
Loopback Pseudo-Interface 1  1 IPv4    {}

PS C:\Users\Administrator>
```

Screenshot 3 – DNS Configuration Restored

Shows the following corrective actions:

```
Set-DnsClientServerAddress -InterfaceAlias "Ethernet0" -ServerAddresses 10.20.20.10
```

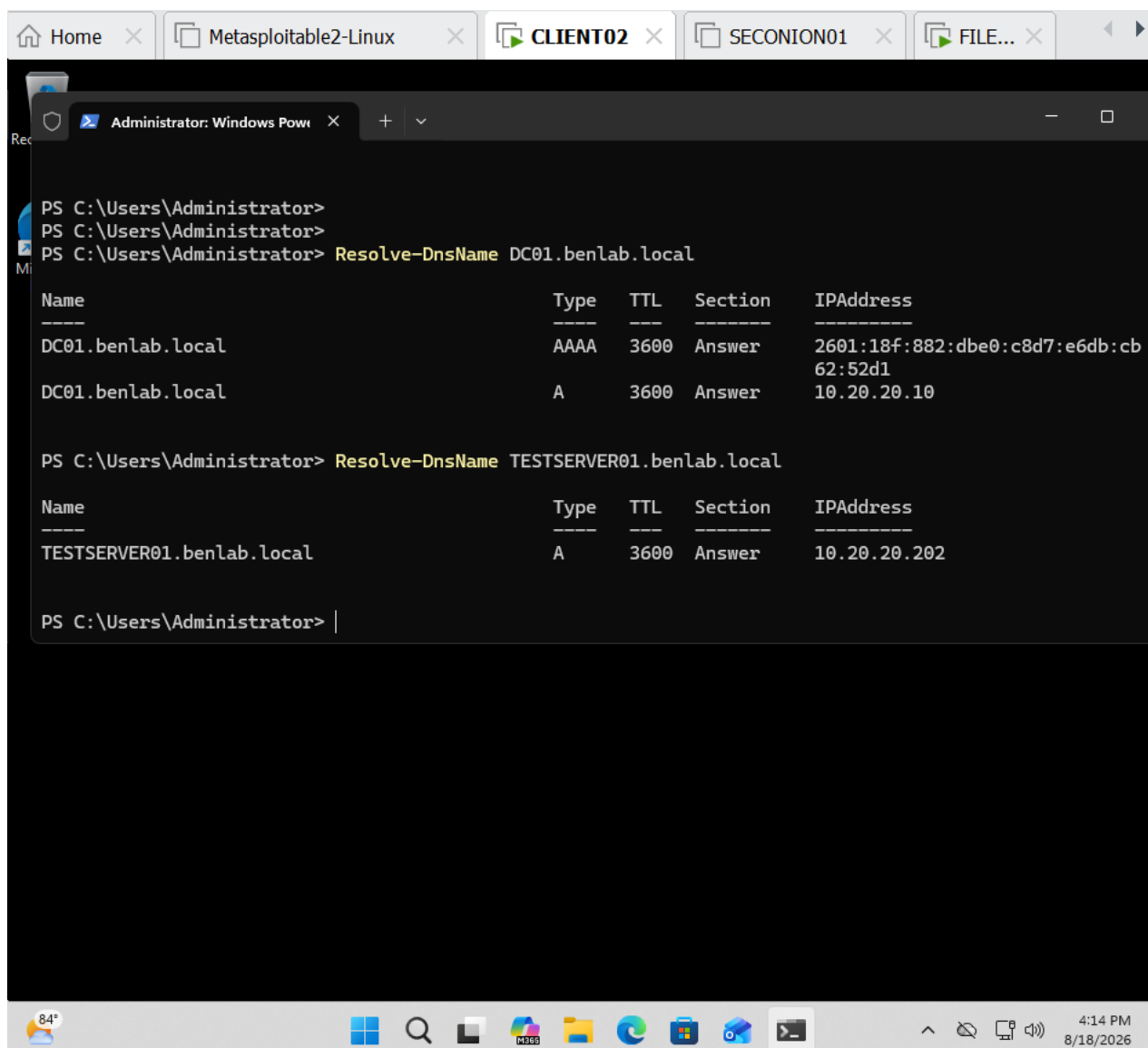
```
Clear-DnsClientCache
```

```
Get-DnsClientServerAddress -AddressFamily IPv4
```

The final output confirms:

```
Ethernet0      IPv4      {10.20.20.10}
```

Purpose: Documents the administrative remediation and verifies that CLIENT02 was restored to the correct internal DNS server.



Screenshot 4 – Successful DNS Resolution

Shows successful execution of:

Resolve-DnsName DC01.benlab.local

Resolve-DnsName TESTSERVER01.benlab.local

with the results:

DC01.benlab.local → 10.20.20.10

TESTSERVER01.benlab.local → 10.20.20.202

Purpose: Provides final evidence that internal DNS name resolution was successfully restored after correcting the client DNS configuration.

Key Takeaways

- Successful IP connectivity does not necessarily mean DNS is functioning correctly.
 - Testing connectivity by IP and hostname separately helps distinguish network problems from DNS problems.
 - Active Directory domain clients should use the organization's internal DNS infrastructure for domain name resolution.
 - Public DNS servers such as 8.8.8.8 cannot resolve private records in the `benlab.local` DNS zone.
 - `Get-DnsClientServerAddress` can identify which DNS servers a Windows client is configured to use.
 - `Resolve-DnsName` is useful for testing forward DNS resolution and diagnosing DNS failures.
 - `Clear-DnsClientCache` can clear cached DNS information after DNS configuration changes.
 - A structured troubleshooting process—**connectivity** → **name resolution** → **DNS configuration** → **remediation** → **verification**—helps isolate DNS problems efficiently.
-

Task 6 – Configure and Verify DNS Conditional Forwarding

Objective

Configure and test DNS conditional forwarding between two DNS servers in the lab. DC01 was configured to forward DNS queries for the `branchlab.local` namespace to APP01, which was configured as the authoritative DNS server for that namespace.

Environment

Primary DNS Server: DC01

DC01 IP Address: 10.20.20.10

Primary Domain: `benlab.local`

Branch DNS Server: APP01

APP01 IP Address: 10.20.20.15

Branch DNS Zone: branchlab.local

Test DNS Record: BRANCHSERV01.branchlab.local

Test IP Address: 10.20.20.100

Procedure

1. Verified DNS configuration on DC01

The existing DNS zones on DC01 were reviewed to verify that the new branch namespace did not already exist.

```
Get-DnsServerZone | Format-Table ZoneName,ZoneType,IsDsIntegrated -AutoSize
```

The results confirmed that DC01 was authoritative for benlab.local and did not already contain a branchlab.local zone.

2. Configured APP01 as a DNS server

The DNS Server role and management tools were installed on APP01.

```
Install-WindowsFeature DNS -IncludeManagementTools
```

The installation completed successfully without requiring a restart.

3. Created the branch DNS zone on APP01

A standalone primary DNS zone named branchlab.local was created on APP01.

```
Add-DnsServerPrimaryZone -Name "branchlab.local" -ZoneFile "branchlab.local.dns"
```

4. Created the branch server DNS record

An A record was created inside the branchlab.local zone:

```
Add-DnsServerResourceRecordA -Name "BRANCHSERV01" -ZoneName "branchlab.local" -IPv4Address "10.20.20.100"
```

The record represents:

```
BRANCHSERV01.branchlab.local → 10.20.20.100
```

5. Verified APP01 DNS resolution directly from DC01

Before configuring conditional forwarding, DC01 was instructed to query APP01 directly:

```
Resolve-DnsName BRANCHSERV01.branchlab.local -Server 10.20.20.15
```

The query successfully returned:

```
BRANCHSERV01.branchlab.local    A    10.20.20.100
```

This confirmed that APP01 was correctly answering DNS queries for the `branchlab.local` namespace.

Troubleshooting Performed

The original design used:

```
branch.benlab.local
```

APP01 successfully hosted the zone and responded to direct DNS queries. However, attempts to configure DC01 with a conditional forwarder for this namespace failed.

PowerShell returned:

```
Failed to create zone branch.benlab.local on server DC01.
```

```
WIN32 9604
```

The DNS namespace design was reviewed. Because `branch.benlab.local` was a child namespace of the existing authoritative `benlab.local` zone, the lab design was changed to use a separate namespace:

```
branchlab.local
```

The original test zone was removed from APP01:

```
Remove-DnsServerZone -Name "branch.benlab.local" -Force
```

The new `branchlab.local` zone and `BRANCHSERV01` A record were then created. This provided a clean environment for demonstrating conditional forwarding between separate DNS namespaces.

Conditional Forwarder Configuration

On DC01, DNS Manager was opened and a new conditional forwarder was configured with:

DNS Domain:

```
branchlab.local
```

Master DNS Server:

```
10.20.20.15
```

The configuration was successfully accepted, and `branchlab.local` appeared under **Conditional Forwarders** in DNS Manager.

The resulting DNS path was:

```
DC01 (10.20.20.10)
```

```
|
| Query for *.branchlab.local
v
Conditional Forwarder
|
v
APP01 (10.20.20.15)
|
| Authoritative for branchlab.local
v
BRANCHSERV01 → 10.20.20.100
```

Verification

The final DNS resolution test was performed on DC01:

```
Resolve-DnsName BRANCHSERV01.branchlab.local
```

Importantly, the `-Server 10.20.20.15` parameter was **not** specified.

DC01 successfully returned:

Name	Type	TTL	Section	IPAddress
----	----	---	-----	-----
BRANCHSERV01.branchlab.local	A	3600	Answer	10.20.20.100

This demonstrated that DC01 automatically matched the `branchlab.local` namespace to its conditional forwarder and sent the DNS query to APP01.

Result

Status: Successful

DC01 was successfully configured to conditionally forward DNS requests for `branchlab.local` to APP01 at `10.20.20.15`.

APP01 successfully resolved `BRANCHSERV01.branchlab.local` to `10.20.20.100`.

This task demonstrated practical administration and troubleshooting of:

- Windows Server DNS roles
- Primary DNS zones
- DNS A records

- Conditional forwarders
- Cross-DNS-server name resolution
- PowerShell DNS administration
- DNS namespace troubleshooting

Screenshot Evidence

The screenshot shows a Windows PowerShell window titled "Administrator: Windows PowerShell" with a black background and white text. The window is open on a desktop with several other windows visible in the background, including "JUICESHOP01", "KALI01", "NESSUS01", "WAZUH01", and "APP01". The PowerShell window displays the following commands and output:

```

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : 
    IPv4 Address. . . . . : 10.20.20.15
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 10.20.20.254

PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Install-WindowsFeatures DNS -IncludeManagementTools
Install-WindowsFeatures : The term 'Install-WindowsFeatures' is not recognized as the name of a cmdlet,
function, script file, or operable program. Check the spelling of the name, or if a path was included,
verify that the path is correct and try again.
At line:1 char:1
+ Install-WindowsFeatures DNS -IncludeManagementTools
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (Install-WindowsFeatures:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\Administrator> Install-WindowsFeature DNS -IncludeManagementTools

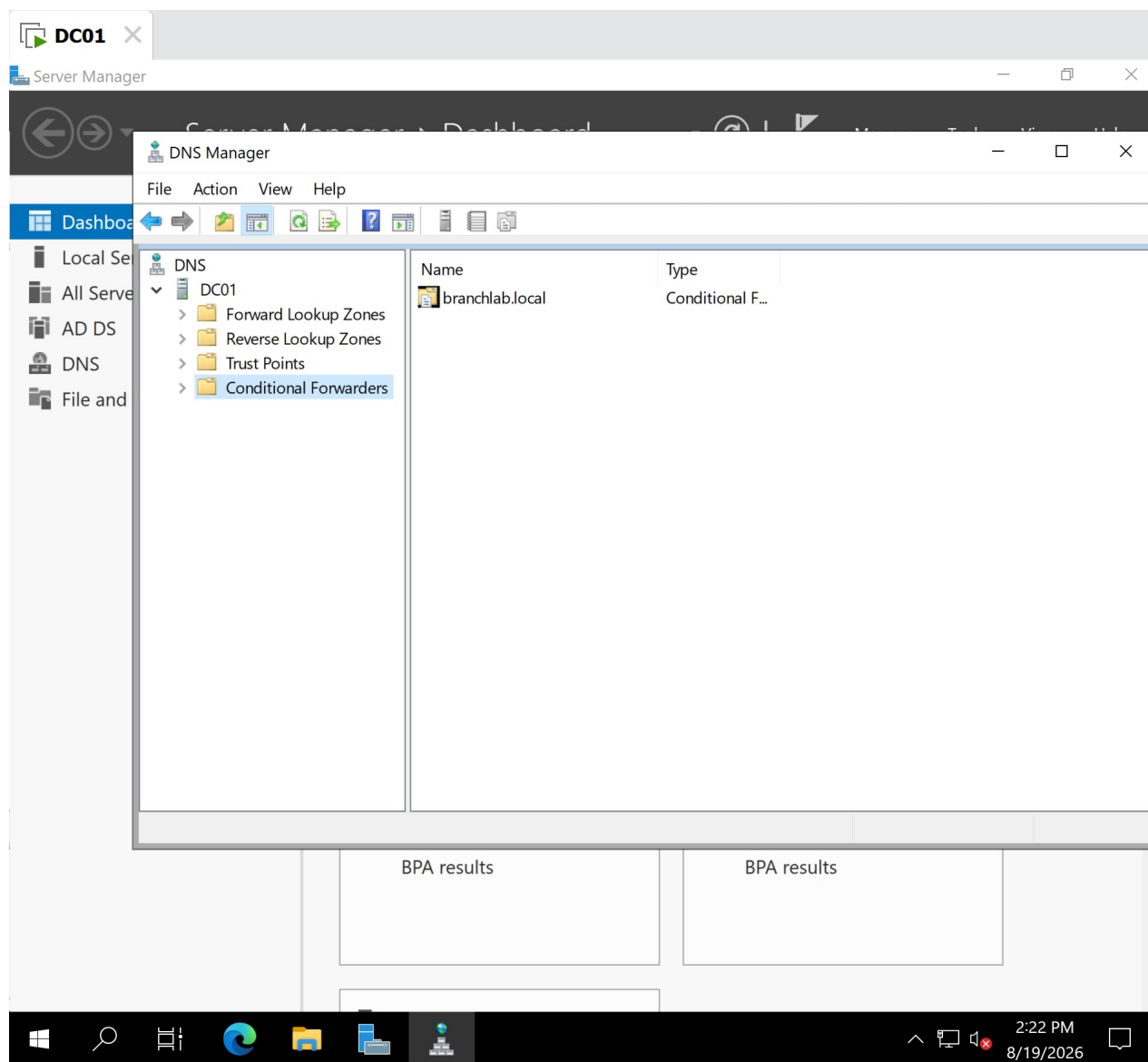
Success Restart Needed Exit Code      Feature Result
-----
True      No          Success      {DNS Server, Remote Server Administration ...

PS C:\Users\Administrator>
  
```

The taskbar at the bottom shows the Windows Start button, a search bar with the text "Type here to search", and several application icons including File Explorer, Edge, and PowerShell. The system clock in the bottom right corner indicates the date and time as "Wednesday, August 19, 2026 10:37 AM 8/19/2026".

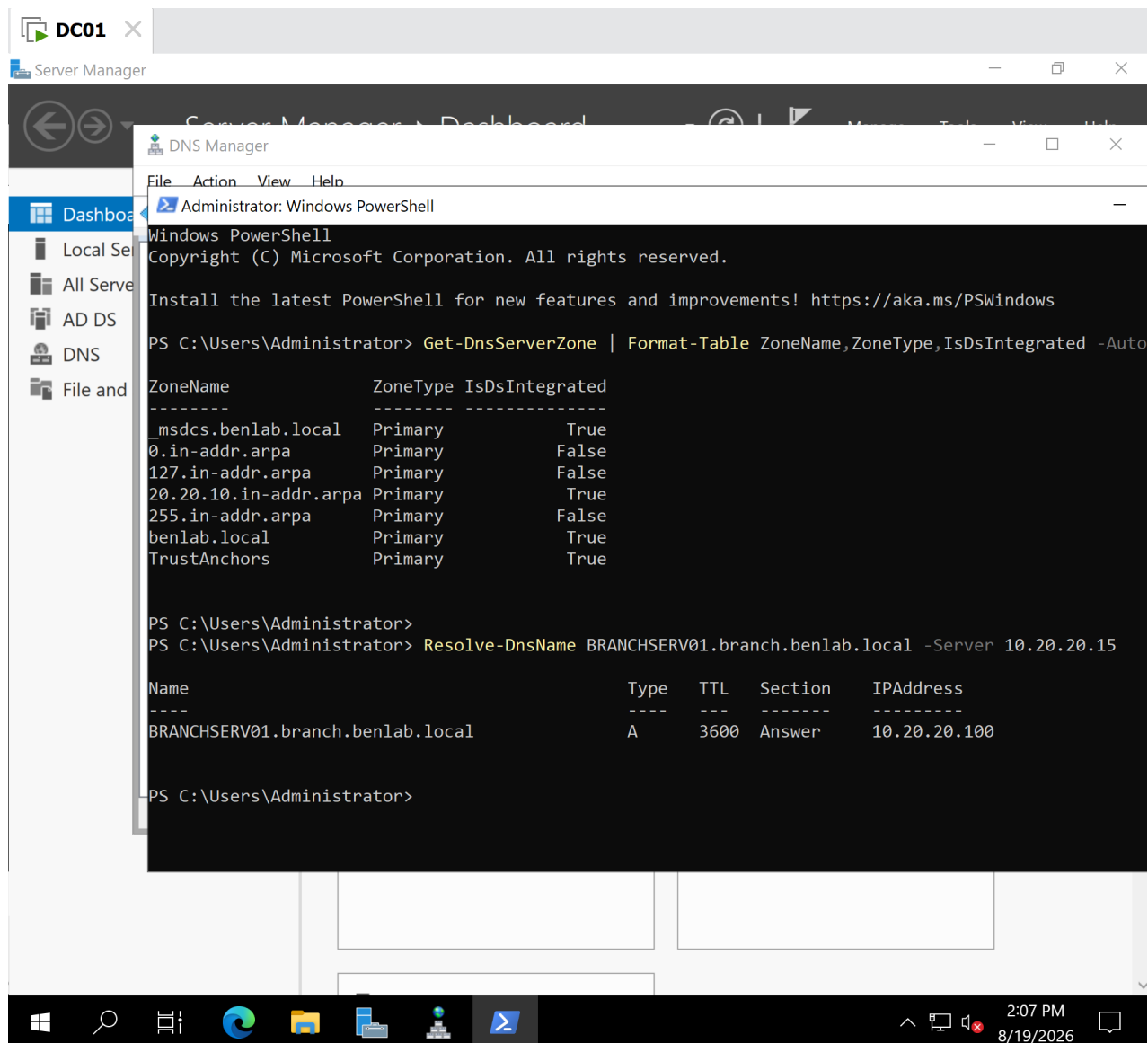
Screenshot 1 – DNS Server Role Installation on APP01

Shows successful installation of the DNS Server role and Remote Server Administration Tools using Install-WindowsFeature.



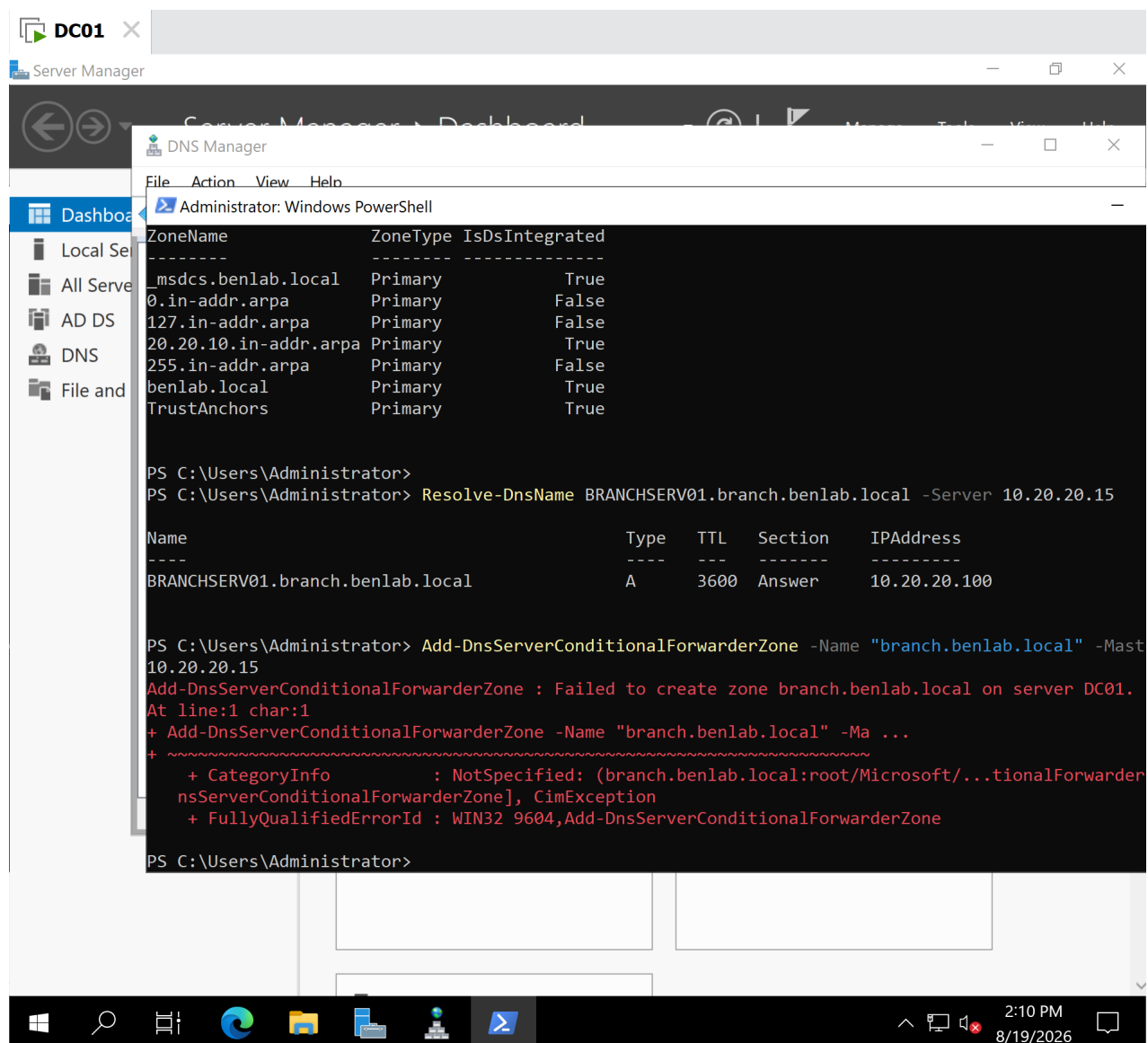
Screenshot 2 – Branch DNS Zone on APP01

Shows the creation and verification of the `branchlab.local` primary DNS zone.



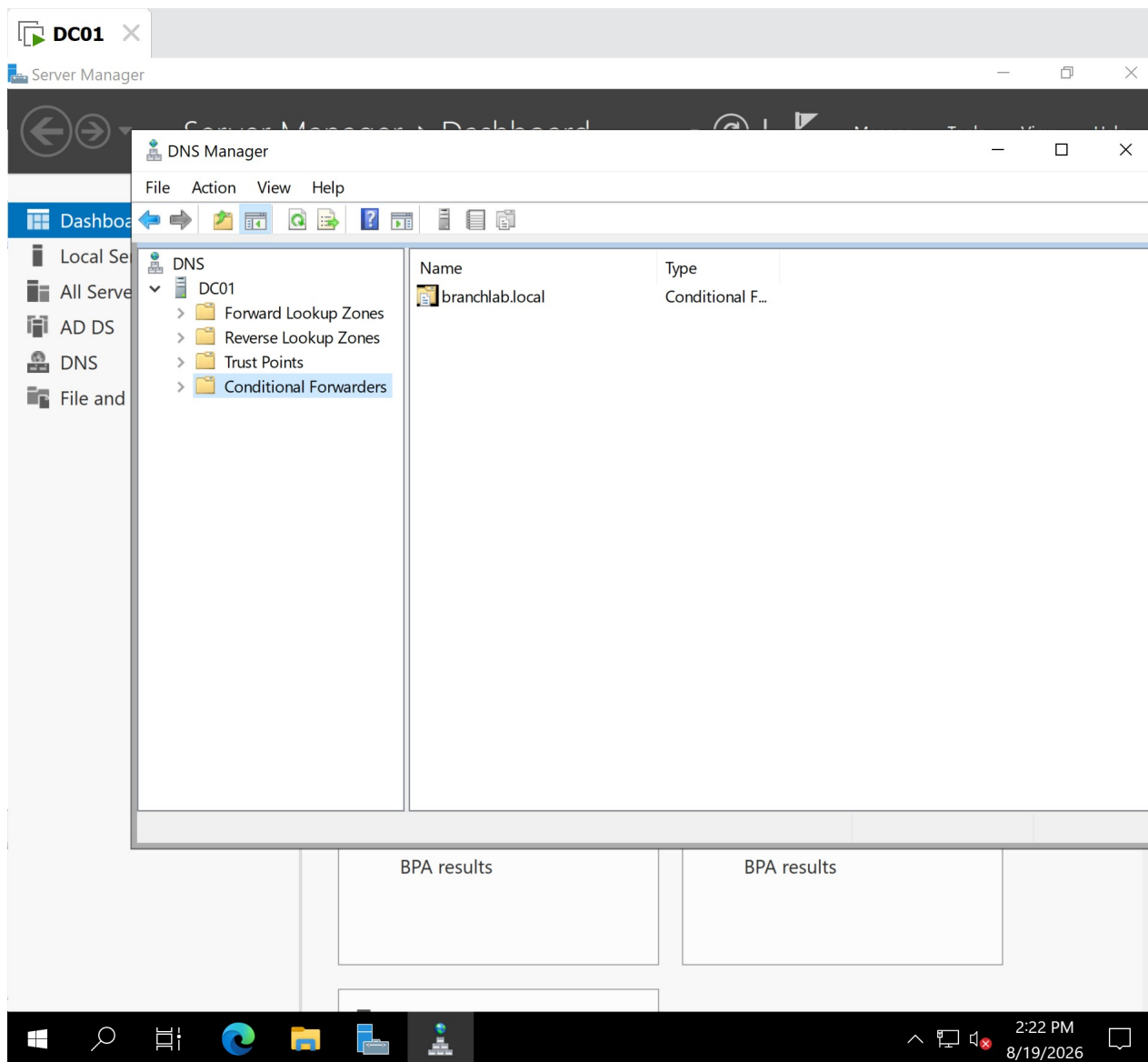
Screenshot 3 – Branch DNS A Record

Shows BRANCHSERV01 configured in the branchlab.local zone with IP address 10.20.20.100.



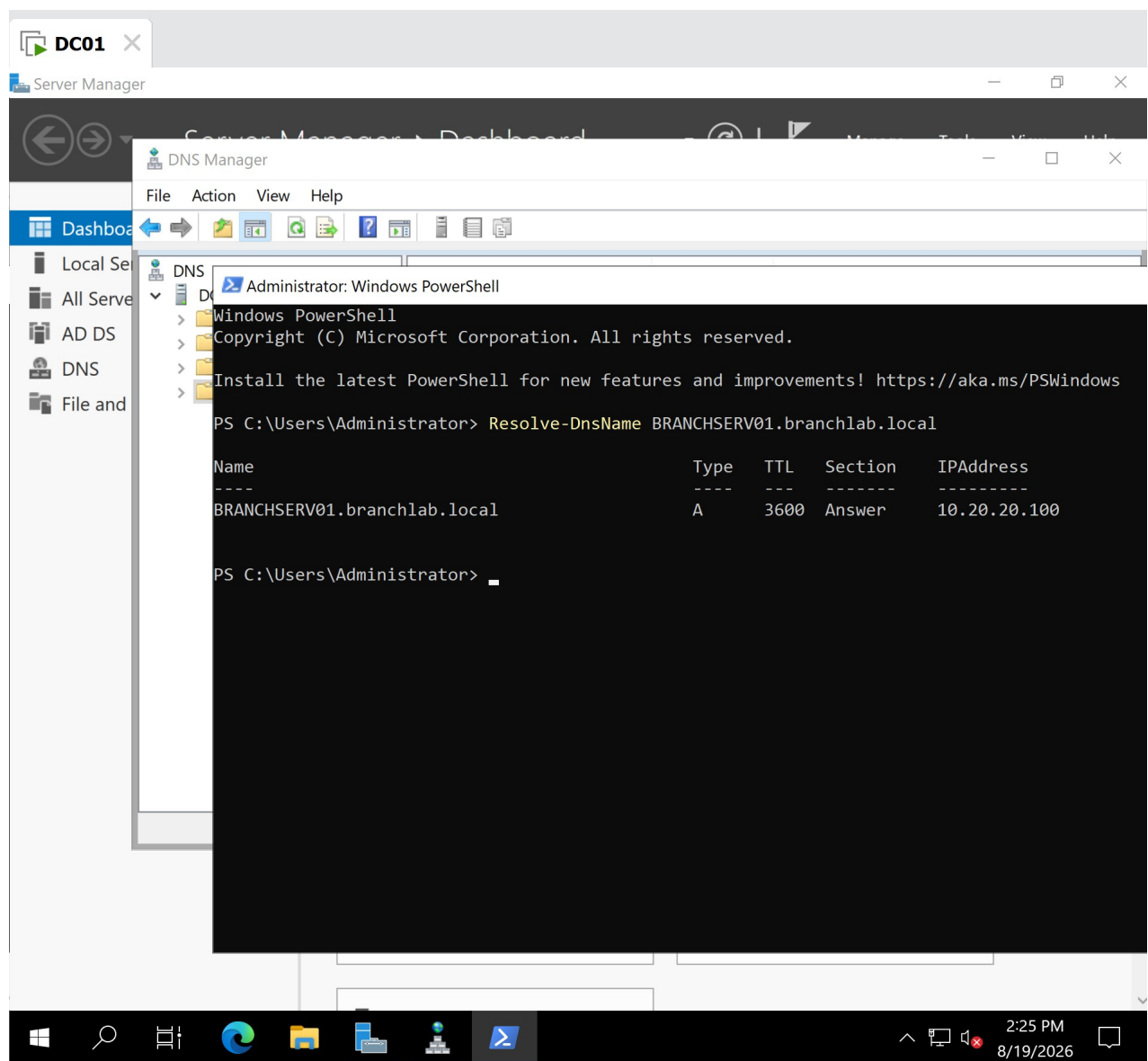
Screenshot 4 – Direct DNS Resolution Test

Shows DC01 successfully resolving `BRANCHSERV01.branchlab.local` directly through APP01 (`10.20.20.15`).



Screenshot 5 – Conditional Forwarder on DC01

Shows branchlab.local listed under **Conditional Forwarders** in DNS Manager on DC01.



Screenshot 6 – Final Conditional Forwarding Verification

Shows:

`Resolve-DnsName BRANCHSERV01.branchlab.local`

successfully returning `10.20.20.100` without explicitly specifying APP01 as the DNS server, confirming that DC01's conditional forwarder is functioning correctly.

Key Learning Outcome

A conditional forwarder allows a DNS server to route queries for a **specific DNS namespace** to a designated DNS server instead of using its normal resolution path. In this lab, DC01 remained responsible for the `benlab.local` environment while forwarding requests for `branchlab.local` to APP01.

The troubleshooting also demonstrated an important administrative principle: when DNS commands and connectivity appear correct but zone creation fails, the **DNS namespace and zone design should also be investigated**, rather than assuming the problem is network connectivity.

Task 7 — DNS Aging and Scavenging

Objective

Configure and verify **DNS Aging and Scavenging** on DC01 to automatically manage stale dynamically registered DNS records in the `benlab.local` zone.

The goal was to understand how Windows DNS distinguishes between **static records** and **dynamic timestamped records**, how records become stale, and how scavenging can safely remove obsolete dynamic DNS records.

Environment

DNS Server: DC01

Domain: `benlab.local`

DNS Zone: `benlab.local`

Operating System: Windows Server 2022

Administration Tools: DNS Manager and Windows PowerShell

Step 1 — Check Existing Zone Aging Configuration

Before making changes, the existing configuration was inspected.

Command

```
Get-DnsServerZoneAging -Name "benlab.local"
```

Initial result

The important values were:

```
ZoneName           : benlab.local
AgingEnabled       : False
RefreshInterval    : 7.00:00:00
```

NoRefreshInterval : 7.00:00:00

Observation

AgingEnabled : False showed that DNS aging was **not enabled** for the benlab.local zone.

Although the default No-Refresh and Refresh intervals existed, the aging mechanism itself was inactive.

Step 2 — Check Server-Level Scavenging

The DNS server's scavenging configuration was then inspected.

Command

```
Get-DnsServerScavenging
```

Initial result

NoRefreshInterval : 7.00:00:00

RefreshInterval : 7.00:00:00

ScavengingInterval : 00:00:00

ScavengingState : False

LastScavengeTime :

Observation

The important result was:

ScavengingState : False

Automatic scavenging was therefore disabled on DC01.

Step 3 — Enable Aging on the DNS Zone

Aging was enabled for benlab.local with a **7-day No-Refresh interval** and a **7-day Refresh interval**.

Command

```
Set-DnsServerZoneAging -Name "benlab.local" -Aging $true -RefreshInterval 7.00:00:00 -NoRefreshInterval 7.00:00:00
```

The configuration was verified with:

```
Get-DnsServerZoneAging -Name "benlab.local"
```

Result

ZoneName : benlab.local
AgingEnabled : True
AvailForScavengeTime : 8/27/2026 6:00:00 PM
RefreshInterval : 7.00:00:00
NoRefreshInterval : 7.00:00:00

AgingEnabled : True confirmed successful configuration.

Step 4 — Enable Server-Level Scavenging

Automatic scavenging was enabled on DC01 with a **7-day scavenging interval**.

Command

```
Set-DnsServerScavenging -ScavengingState $true -ScavengingInterval 7.00:00:00
```

Verification:

```
Get-DnsServerScavenging
```

Result

NoRefreshInterval : 7.00:00:00
RefreshInterval : 7.00:00:00
ScavengingInterval : 7.00:00:00
ScavengingState : True
LastScavengeTime :

The important result was:

ScavengingState : True

Automatic DNS scavenging was successfully enabled.

Step 5 — Examine DNS Record Timestamps

The records inside `benlab.local` were inspected to determine which records were static and which contained aging timestamps.

Command

```
Get-DnsServerResourceRecord -ZoneName "benlab.local" |  
Select-Object HostName, RecordType, Timestamp, RecordData |  
Format-Table -AutoSize
```


Several records contained timestamps, including records associated with systems such as:

APP01

FILESRV01

WIN01

DESKTOP-0499M0A

Other records had no timestamp.

This demonstrated that **not every DNS record is treated the same way by aging and scavenging**.

Step 6 — Compare Dynamic and Static Records

Two records were examined individually.

FILESRV01

```
Get-DnsServerResourceRecord -ZoneName "benlab.local" -Name "FILESRV01" |
```

```
Format-List HostName,RecordType,Timestamp,RecordData
```

Result included:

HostName : FILESRV01

RecordType : A

Timestamp : 6/20/2026 1:00:00 PM

FILESRV01 therefore had a timestamp associated with the record.

TESTSERVER01

```
Get-DnsServerResourceRecord -ZoneName "benlab.local" -Name "TESTSERVER01" |
```

```
Format-List HostName,RecordType,Timestamp,RecordData
```

Result:

HostName : TESTSERVER01

RecordType : A

Timestamp :

The empty timestamp indicated that TESTSERVER01 was a **static record**.

Step 7 — Verify Records in DNS Manager

DNS Manager was opened and **Advanced View** was enabled so that the **Timestamp** column could be inspected.

The `benlab.local` zone showed examples such as:

Record	IP Address	Timestamp
dc01	10.20.20.10	static
APP01	10.20.20.15	timestamped
FILESRV01	10.20.20.50	timestamped
TESTSERVER01	10.20.20.202	static
WIN01	10.20.20.140	timestamped

This provided GUI confirmation of the same information observed through PowerShell.

Final Verification

The complete configuration was verified one final time.

Zone Aging

```
Get-DnsServerZoneAging -Name "benlab.local"
```

Confirmed:

```
AgingEnabled           : True
RefreshInterval        : 7.00:00:00
NoRefreshInterval      : 7.00:00:00
```

Server Scavenging

```
Get-DnsServerScavenging
```

Confirmed:

```
ScavengingInterval : 7.00:00:00
ScavengingState    : True
```

Final Status

DNS Aging: Enabled 

No-Refresh Interval: 7 days 

Refresh Interval: 7 days 

DNS Scavenging: Enabled 

Scavenging Interval: 7 days 

No manual scavenging operation was forced during the task.

Key Concepts Learned

1. DNS Aging

Aging allows Windows DNS to track how old eligible dynamically registered records are by using **timestamps**.

It does not itself delete records.

2. No-Refresh Interval

For the first **7 days**, Windows does not need to update the timestamp merely because the client refreshes the same record.

This helps reduce unnecessary DNS database updates and AD replication traffic for AD-integrated zones.

3. Refresh Interval

After the No-Refresh interval, the record enters the **Refresh interval**.

During this period, an eligible client can refresh its DNS registration and update the timestamp.

If the record continues to be refreshed appropriately, it remains active.

4. Scavenging

Scavenging is the cleanup mechanism.

It examines records that have become stale according to the aging configuration and can remove eligible stale dynamic records.

The conceptual process is:

Dynamic DNS Record

↓

Timestamp

↓

No-Refresh Interval

↓

Refresh Interval

↓

Record not refreshed

↓

Record becomes stale



Scavenging



Stale record removed

Static vs. Dynamic DNS Records

One of the most important lessons from this task was understanding that scavenging does **not simply delete records because their IP addresses are old**.

A static record such as:

`dc01 → 10.20.20.10 → static`

does not have the same aging timestamp behavior as a dynamically registered record such as:

`WIN01 → 10.20.20.140 → timestamped`

This is why enabling scavenging does not mean Windows will simply start deleting important manually configured server records such as the static A record for `DC01`.

Real-World Importance

Over time, client computers can receive different IP addresses, be renamed, be removed from the environment, or stop connecting altogether.

Without proper DNS maintenance, obsolete records can accumulate.

For example:

`WIN01 → 10.20.20.140`

If that dynamically registered system disappears permanently and its record is never refreshed, the DNS record can eventually become stale.

Aging and scavenging provide Windows administrators with an automated mechanism for cleaning up these eligible stale DNS records.

This helps maintain a **cleaner and more accurate DNS database**.

PowerShell Commands Used

```
Get-DnsServerZoneAging -Name "benlab.local"
```

Get-DnsServerScavenging

```
Set-DnsServerZoneAging -Name "benlab.local" -Aging $true -RefreshInterval 7.00:00:00 -NoRefreshInterval 7.00:00:00
```

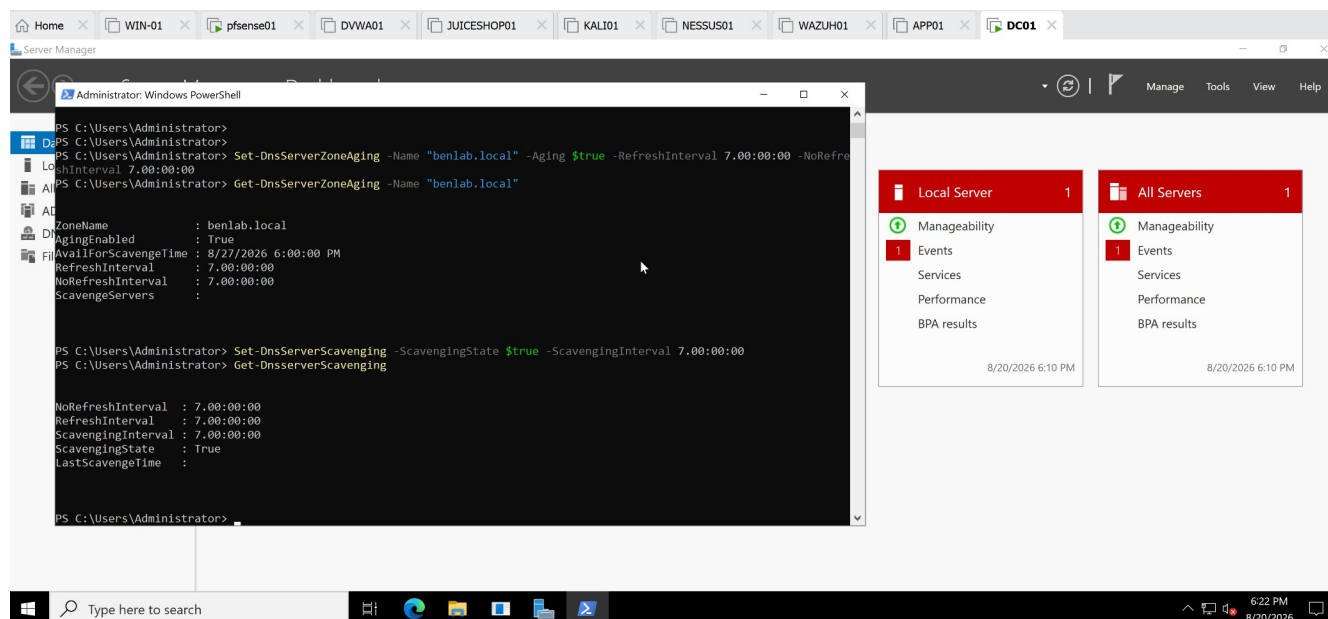
```
Set-DnsServerScavenging -ScavengingState $true -ScavengingInterval 7.00:00:00
```

```
Get-DnsServerResourceRecord -ZoneName "benlab.local" |  
Select-Object HostName, RecordType, Timestamp, RecordData |  
Format-Table -AutoSize
```

```
Get-DnsServerResourceRecord -ZoneName "benlab.local" -Name "FILESRV01" |  
Format-List HostName,RecordType,Timestamp,RecordData
```

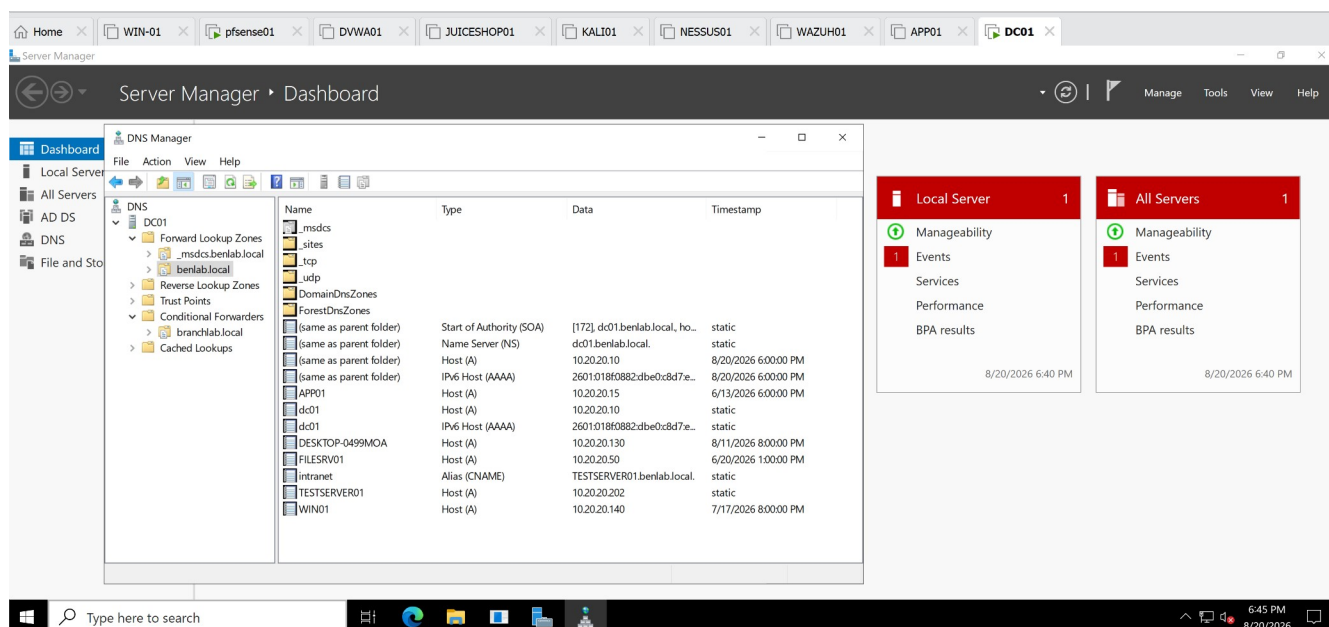
```
Get-DnsServerResourceRecord -ZoneName "benlab.local" -Name "TESTSERVER01" |  
Format-List HostName,RecordType,Timestamp,RecordData
```

Screenshots Evidence



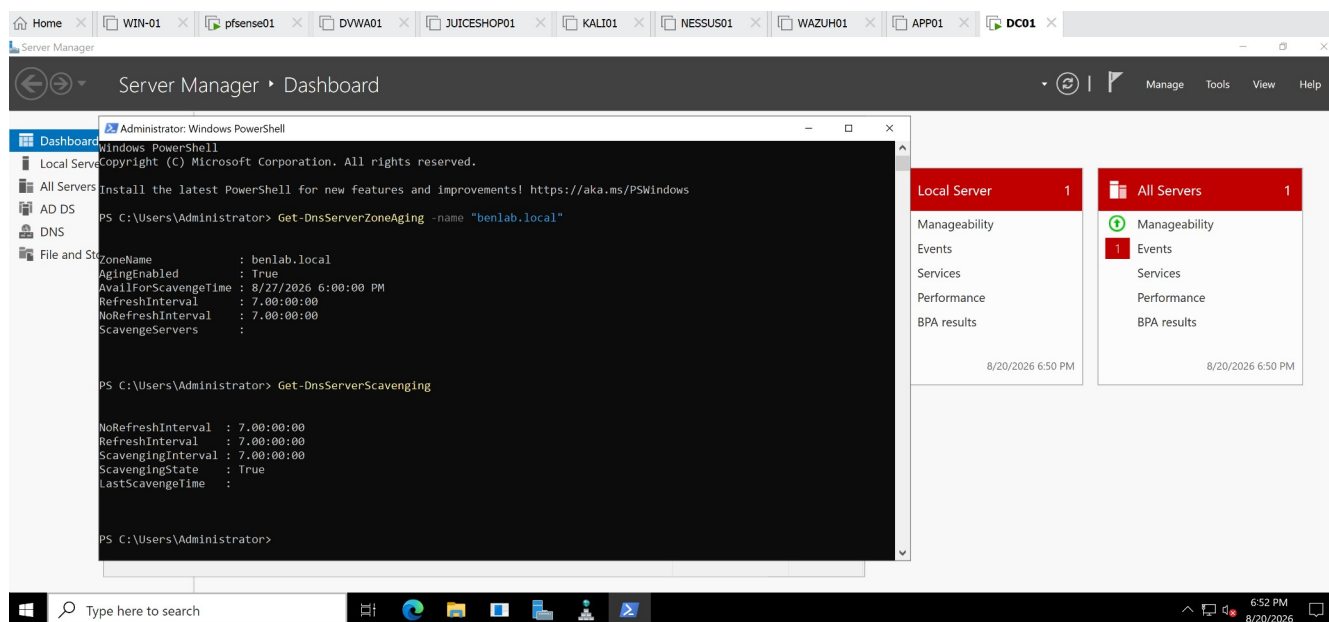
Screenshot 1 — Aging Configuration

PowerShell showing AgingEnabled : True, along with the 7-day Refresh and No-Refresh intervals.



Screenshot 2 — Static vs. Timestamped Records

DNS Manager Advanced View showing dc01 and TESTSERVER01 as **static**, while APP01, FILESRV01, and WIN01 contain timestamps.



Screenshot 3 — Final Verification

AgingEnabled : True

ScavengingState : True

Module 3 – DNS Administration

Task 8 – DNS Cache Management and Client-Side DNS Validation

Objective

The objective of this task was to examine and manage DNS caching at both the **DNS server** and **DNS client** levels.

The task demonstrated how:

- DC01 maintains a DNS server cache.
 - WIN01 maintains its own local DNS resolver cache.
 - DNS queries for internal and external resources are resolved.
 - Cached DNS entries can be viewed and cleared.
 - DNS connectivity problems can affect name resolution.
-

Environment

System	Role	IP Address
DC01	Domain Controller / DNS Server	10.20.20.10
WIN01	Domain-joined Windows 11 client	10.20.20.140
pfSense01	Gateway / DHCP	10.20.20.254
Domain	Active Directory domain	benlab.local

WIN01 was configured to use:

10.20.20.10

as its IPv4 DNS server, meaning DNS queries from WIN01 are sent to **DC01**.

Part 1 – Examining the DNS Server Cache on DC01

On DC01, the DNS server cache configuration was examined using:

Get-DnsServerCache

The output displayed DNS server cache properties including:

MaxTTL

MaxNegativeTTL

EnablePollutionProtection

LockingPercent

This confirmed that the Microsoft DNS service on DC01 maintains a cache for previously resolved DNS information.

External DNS Resolution Test

An external DNS query was generated with:

```
Resolve-DnsName microsoft.com
```

The query successfully returned DNS records for `microsoft.com`, including IPv4 and IPv6 addresses.

This demonstrated that DC01 could resolve an external Internet domain.

Part 2 – Viewing Cached Records on DC01

The DNS server cache was inspected for Microsoft-related records using:

```
Get-DnsServerResourceRecord -ZoneName "." |
```

```
Where-Object {$_.Hostname -like "*microsoft*"} |
```

```
Format-Table Hostname,RecordType,Timestamp,RecordData -AutoSize
```

Multiple Microsoft-related records appeared in the DNS server cache, including record types such as:

A

AAAA

CNAME

NS

This demonstrated that external DNS information obtained during DNS resolution can be temporarily stored by the DNS server.

Part 3 – Clearing the DNS Server Cache

The DNS server cache on DC01 was cleared using:

```
Clear-DnsServerCache -Force
```

The Microsoft-related cache query was then executed again.

The previously cached records were no longer displayed.

Key concept

Clear-DnsServerCache clears **temporary cached DNS information**.

It does **not** delete authoritative DNS records from the benlab.local DNS zone, such as:

DC01 → 10.20.20.10

Therefore:

DNS cache records and DNS zone records are not the same thing.

Part 4 – Examining the WIN01 DNS Client Cache

On WIN01, the local Windows DNS resolver cache was examined using:

```
ipconfig /displaydns
```

The output showed cached DNS records from previous DNS queries.

Because the output can contain many records, this demonstrated how quickly a Windows workstation accumulates temporary DNS information during normal operation.

Part 5 – WIN01 Network Troubleshooting

During the task, WIN01 unexpectedly received an APIPA address:

169.254.x.x

Instead of receiving an address from the pfSense DHCP scope:

10.20.20.100 – 10.20.20.200

Connectivity to the pfSense gateway also failed:

```
ping 10.20.20.254
```

with:

```
PING: transmit failed. General failure.
```

The network adapter itself was checked with:

```
Get-NetAdapter
```

and appeared **Up**, indicating that Windows recognized the virtual NIC even though the TCP/IP configuration was malfunctioning.

Part 6 – DHCP and TCP/IP Troubleshooting

Attempts were made to obtain a new DHCP lease:

```
ipconfig /release
```

```
ipconfig /renew
```

The commands returned network endpoint/NCB errors rather than obtaining a valid lease.

The Windows networking stack was therefore reset using:

```
netsh winsock reset
```

followed by:

```
netsh int ip reset
```

Windows reported that a restart was required.

After restarting WIN01, DHCP communication was restored and WIN01 successfully obtained:

```
IPv4 Address : 10.20.20.140
```

```
Subnet Mask : 255.255.255.0
```

```
Gateway : 10.20.20.254
```

```
DHCP Server : 10.20.20.254
```

Connectivity tests to both the gateway and DC01 were successful.

Part 7 – Clearing the WIN01 DNS Client Cache

The workstation's local DNS cache was cleared using:

```
ipconfig /flushdns
```

Windows confirmed:

Successfully flushed the DNS Resolver Cache.

Running:

```
ipconfig /displaydns
```

immediately afterward showed that the previously accumulated cache entries had been removed.

Part 8 – Internal DNS Resolution

WIN01 was then used to query the domain controller:

```
Resolve-DnsName DC01.benlab.local
```

The query successfully returned the internal IPv4 record:

DC01.benlab.local

A

10.20.20.10

This verified that WIN01 could communicate with the internal DNS server and resolve resources in the benlab.local namespace.

Part 9 – External DNS Resolution and Client Caching

An external DNS lookup was performed from WIN01:

```
Resolve-DnsName microsoft.com
```

The query returned an IPv4 **A record** and IPv6 **AAAA record** for microsoft.com.

WIN01's DNS cache was then inspected again:

```
ipconfig /displaydns
```

The newly resolved microsoft.com information appeared in the local client cache.

This demonstrated the DNS resolution path:

WIN01

|

| DNS query

▼

DC01 (10.20.20.10)

|

| resolves external name

▼

External DNS infrastructure

|

▼

DC01

|

| DNS response

▼

WIN01

|



Stores result in local DNS cache

Once the IP address is known, normal application traffic can then travel from WIN01 through pfSense toward the destination. DC01 is providing **DNS resolution**, not acting as the router for the subsequent web connection.

Key Learning Points

1. DNS servers and DNS clients maintain separate caches.

DC01 maintains the server-side DNS cache, while WIN01 maintains its own local DNS resolver cache.

2. `clear-DnsServerCache` and `ipconfig /flushdns` perform different jobs.

`clear-DnsServerCache` clears DC01's DNS server cache, while `ipconfig /flushdns` clears WIN01's local resolver cache.

3. Clearing DNS cache does not delete authoritative DNS zone records.

Records belonging to the `benlab.local` zone remain intact.

4. DNS resolution depends on network connectivity.

WIN01's APIPA condition prevented normal communication with pfSense and DC01 even though the network adapter itself showed as Up.

5. An APIPA address (`169.254.x.x`) indicates that the client failed to obtain a usable IPv4 configuration from DHCP.

6. Resetting Winsock/TCP-IP and restarting WIN01 restored DHCP connectivity in this troubleshooting case.

7. Internal and external DNS resolution use the same client DNS infrastructure.

WIN01 can ask DC01 about both:

`DC01.benlab.local`

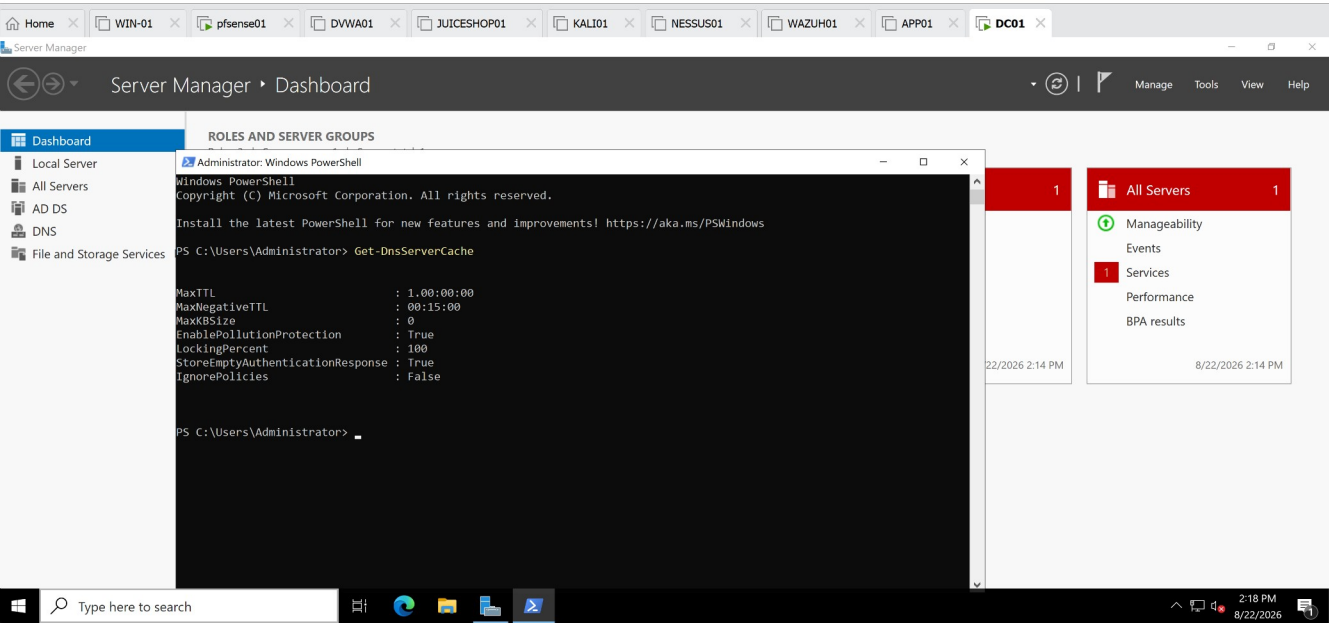
`microsoft.com`

but DC01 is authoritative for `benlab.local` while external names must be resolved through DNS outside that authoritative zone.

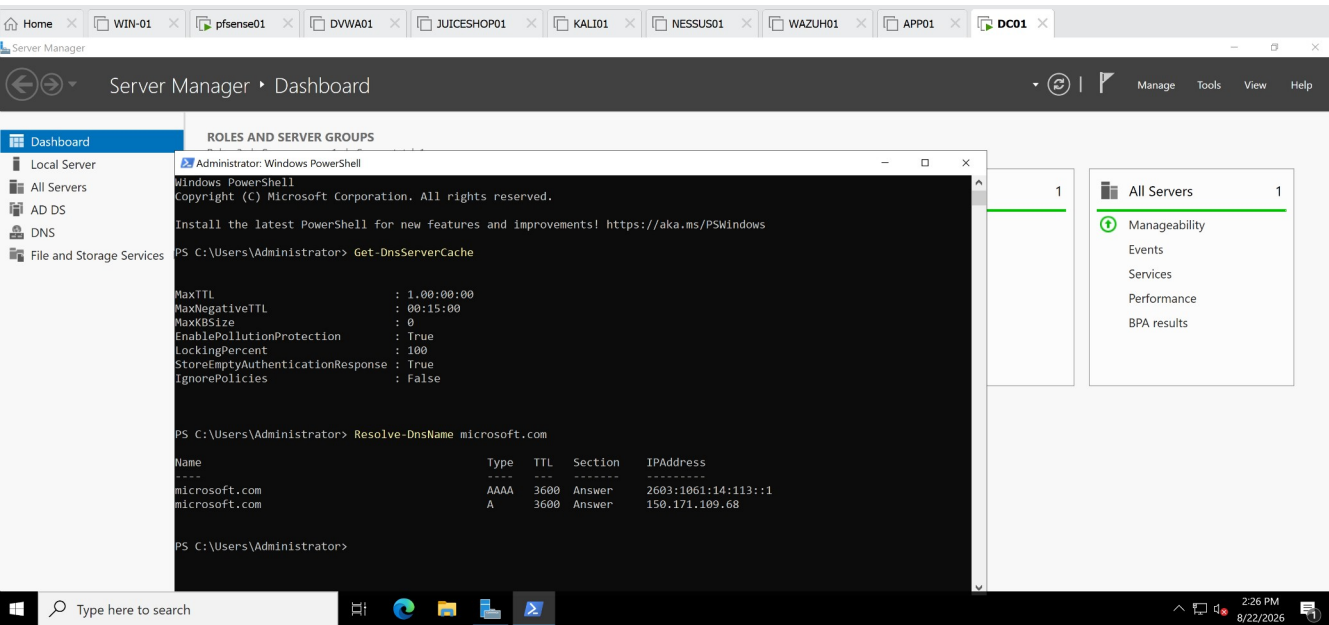
8. DNS resolution and actual application traffic are different processes.

DNS tells WIN01 the destination IP address; after resolution, WIN01 communicates with that destination through the network rather than sending all application traffic through DC01.

Screenshots Evidence

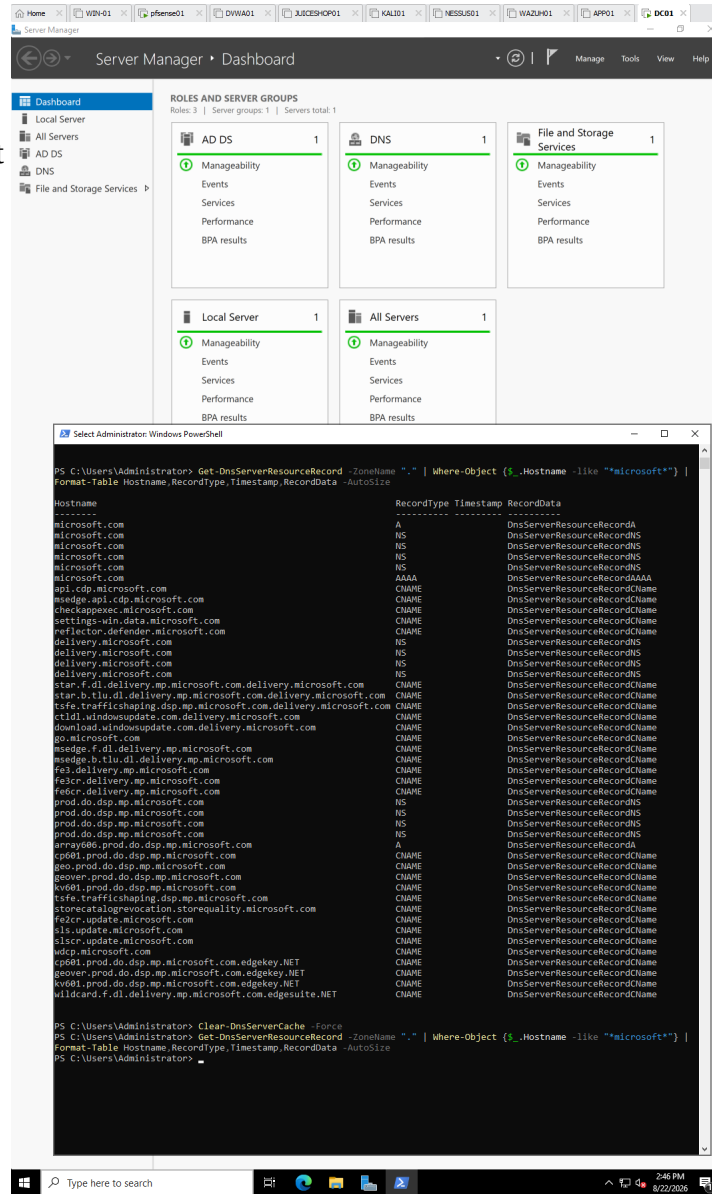


Screenshot 1: Get -DnsServerCache on DC01

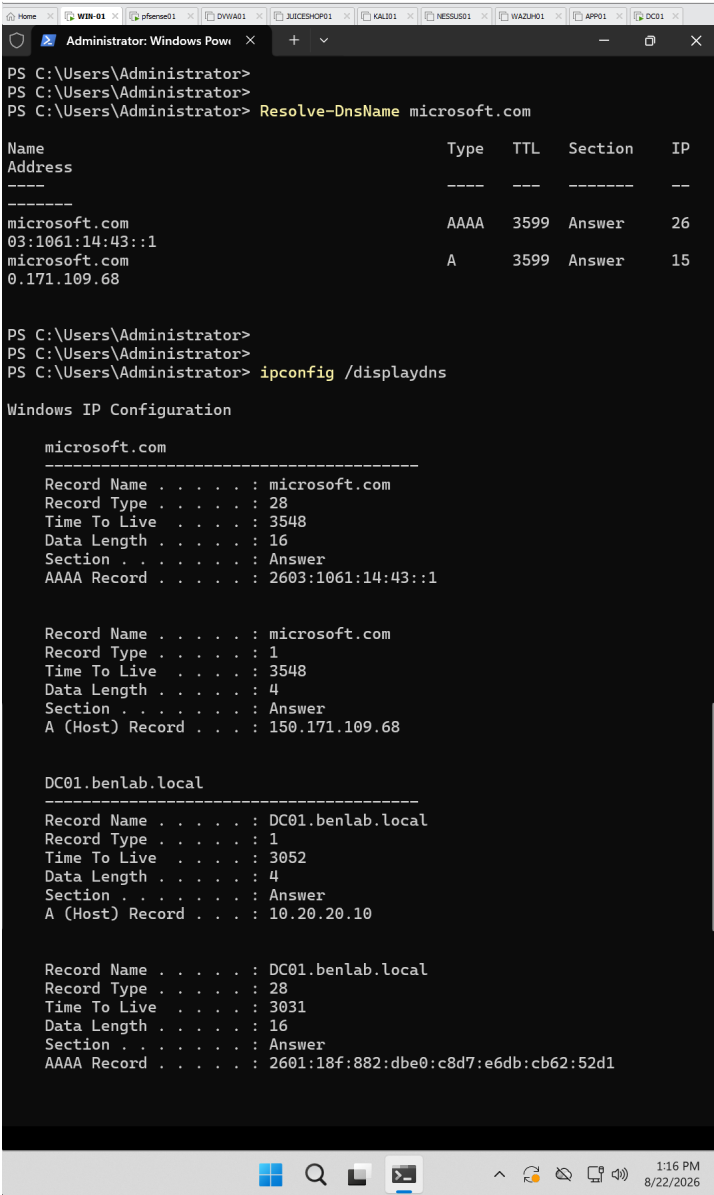


Screenshot 2: Microsoft records visible in DC01's DNS server cache

Screenshot 3: Cache cleared with Clear-DnsServerCache -Force and Microsoft records disappearing



Screenshot 4: WIN01
showing Resolve-DnsName
microsoft.com
followed by
ipconfig
/displaydns



Task 9 — DNS Event Log Monitoring and Health Verification

Objective

Monitor the DNS Server event logs on **DC01**, identify DNS-related warnings or errors, determine whether the events indicate an active DNS problem, and verify the current operational health of the DNS service.

Environment

Server: DC01

Domain: benlab.local

DC01 IPv4: 10.20.20.10

Roles involved: Active Directory Domain Services (AD DS) and DNS Server

Step 1 — Open the DNS Server Event Log

On **DC01**, opened:

Server Manager → **Tools** → **Event Viewer**

Then navigated to:

Applications and Services Logs → **DNS Server**

The DNS Server log contained both informational events and warnings generated by the DNS Server service.

Step 2 — Filter Significant DNS Events

Used **Filter Current Log** and selected:

- Critical
- Error
- Warning

This reduced the DNS log to **62 significant events** requiring investigation.

The events were then examined by **Event ID**.

Step 3 — Investigate Event ID 4013

The filtered log showed repeated:

Source: DNS-Server-Service

Event ID: 4013

Level: Warning

Event 4013 indicated that the DNS Server service was waiting for **Active Directory Domain Services (AD DS)** to complete its initial directory synchronization before DNS could fully initialize.

Because DC01 hosts both **AD DS and DNS**, DNS depends on Active Directory for its AD-integrated DNS data.

Step 4 — Verify the Event Count

The DNS Server log was filtered specifically for:

4013

The result remained:

Number of events: 62

Therefore:

All 62 Critical/Error/Warning events identified during the investigation were Event ID 4013 warnings.

No separate collection of DNS critical errors or other DNS failure event IDs was identified in this filtered set.

Step 5 — Verify DNS Server Service Status

Opened **PowerShell as Administrator** on DC01 and ran:

```
Get-Service DNS
```

Result:

Status	Name	DisplayName
Running	DNS	DNS Server

This confirmed that the **DNS Server service was currently running.**

Step 6 — Run Domain Controller DNS Diagnostics

Ran:

```
dcdiag /test:dns
```

The diagnostic returned successful results including:

```
DC01 passed test Connectivity
```

```
DC01 passed test DNS
```

```
benlab.local passed test DNS
```

This verified that DC01 passed its Domain Controller connectivity and DNS diagnostic testing.

Results

The investigation determined that the DNS Server service on **DC01** was **operational and healthy at the time of testing**.

Although Event Viewer contained **62 warning events**, all 62 were **Event ID 4013**, associated with AD DS/DNS initialization and synchronization.

Current health verification showed:

```
DNS Server Service      → Running
```

```
DC01 Connectivity Test  → Passed
```

```
DC01 DNS Test           → Passed
```

```
benlab.local DNS Test   → Passed
```

The Event 4013 warnings therefore did **not provide evidence of a current DNS outage**. They were correlated with the server's AD DS/DNS startup synchronization behavior.

Key Learning Points

Event Viewer is historical. A warning recorded in Event Viewer does not necessarily mean that the problem is occurring right now.

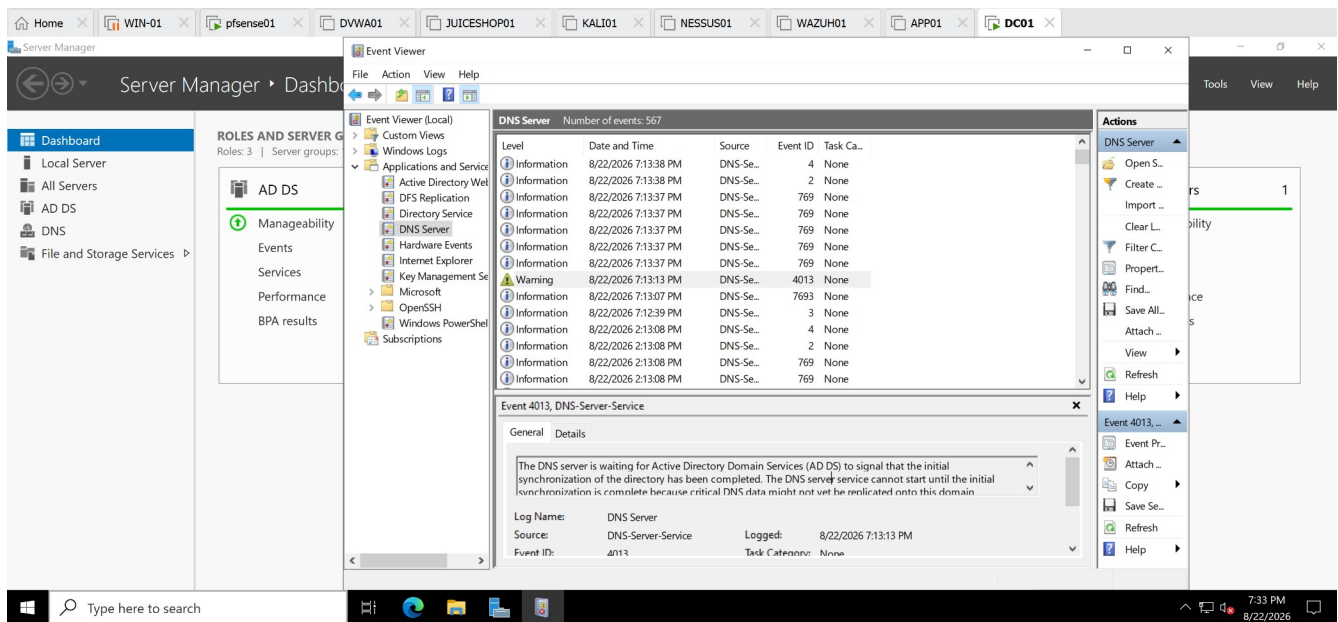
Event IDs require investigation. Administrators should examine the Event ID, source, timestamp, description, frequency, and current server behavior before deciding that something needs to be repaired.

Service status alone is not enough. Get-Service DNS verified that DNS was running, while dcdiag /test:dns provided deeper validation that the Domain Controller's DNS functionality passed diagnostic testing.

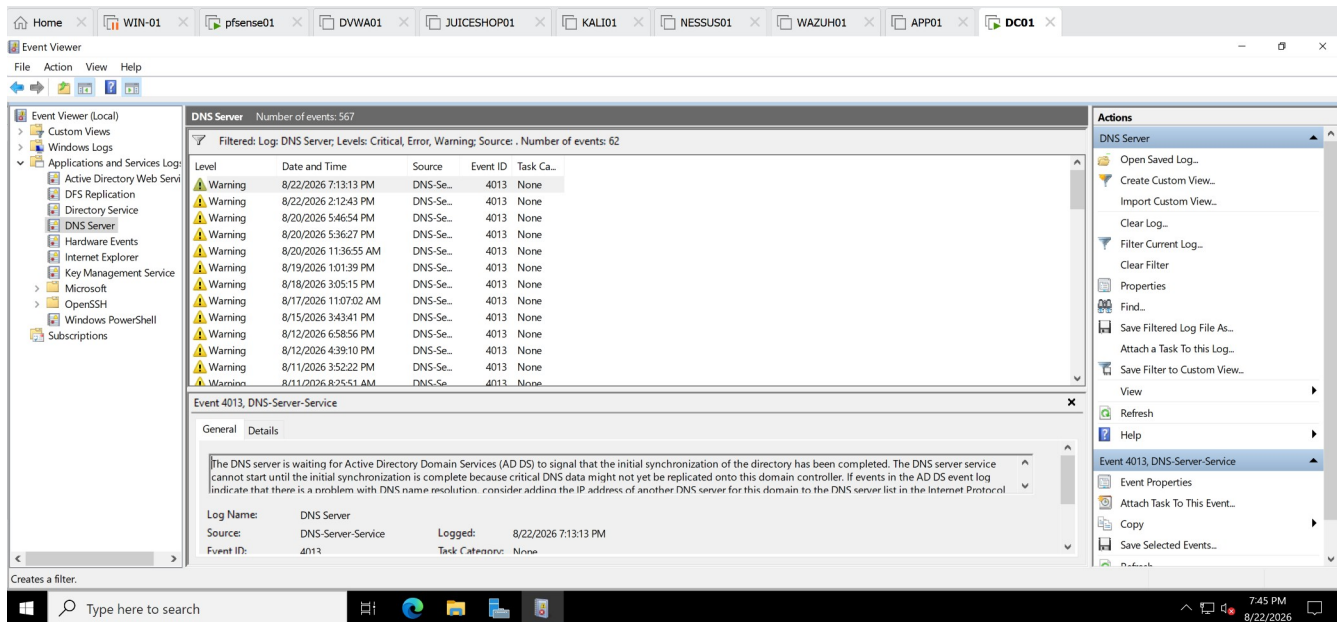
AD DS and DNS are closely integrated. On DC01, DNS hosts Active Directory-integrated DNS information, which explains why DNS may wait for AD DS during initialization.

Warnings should be correlated with operational testing. In this task, Event Viewer showed Event ID 4013 warnings, while current DNS diagnostics passed successfully.

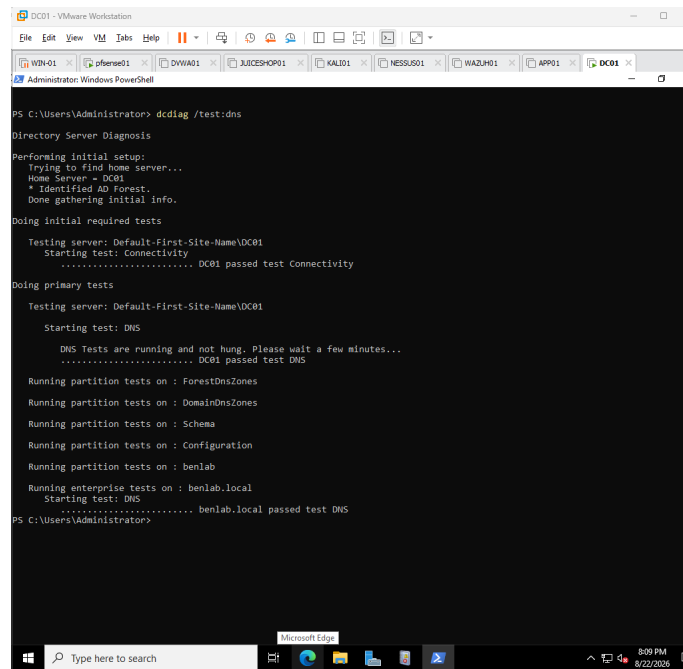
Screenshots



Event Viewer filtered to Critical/Error/Warning, showing Event ID 4013.



Event Viewer filtered specifically to 4013, showing Number of events: 62.



```
PS C:\Users\Administrator> dcdiag /test:dns
Directory Server Diagnosis

Performing initial setup:
  Trying to find home server...
  Home Server - DC01
  Identified AD Forest.
  Done gathering initial info.

Doing initial required tests

Testing server: Default-First-Site-Name\DC01
  Starting test: Connectivity
  ..... DC01 passed test Connectivity

Doing primary tests

Testing server: Default-First-Site-Name\DC01
  Starting test: DNS
  DNS Tests are running and not hung. Please wait a few minutes...
  ..... DC01 passed test DNS

Running partition tests on : ForestDnsZones
Running partition tests on : DomainDnsZones
Running partition tests on : Schema
Running partition tests on : Configuration
Running partition tests on : benlab
Running enterprise tests on : benlab.local
  Starting test: DNS
  ..... benlab.local passed test DNS
PS C:\Users\Administrator>
```

PowerShell DNS health verification, especially the screenshot showing:

DC01 passed test DNS

benlab.local passed test DNS

Module 3 – DNS Administration

Task 5 – DNS Troubleshooting and Validation

Objective

Perform systematic DNS troubleshooting and validation within the `benlab.local` Active Directory environment.

The purpose of this task was to verify DNS service health, DNS zones and resource records, forward and reverse name resolution, Active Directory service discovery, Kerberos and LDAP connectivity, Domain Controller discovery, external DNS resolution, DNS client caching, and overall DNS health.

Environment

System	Role	IP Address
DC01	Domain Controller / AD DS / DNS Server	10.20.20.10
WIN01	Domain-joined Windows workstation	10.20.20.140
FILESRV01	Windows File Server	10.20.20.50
pfSense01	Gateway / DHCP Server	10.20.20.254
Domain	Active Directory Domain	benlab.local

Troubleshooting was performed primarily from **DC01** and **WIN01** using Windows PowerShell and command-line administration tools.

Troubleshooting and Validation Steps

Step 1 – Verify the DNS Server Service

On DC01, verify that the Windows DNS Server service is running.

Command

```
Get-Service DNS
```

Expected/Observed Result

Status	Name	DisplayName
Running	DNS	DNS Server

Explanation

This verifies that the DNS Server service itself is operational.

Checking the service is an important early troubleshooting step because DNS queries cannot be processed normally if the DNS service is stopped.

Result

Successful — DNS Server service was running on DC01.

Step 2 – Verify DNS Zones

List the DNS zones configured on DC01.

Command

```
Get-DnsServerZone
```

The output confirmed important zones including:

`_msdcs.benlab.local`

`20.20.10.in-addr.arpa`

`benlab.local`

`branchlab.local`

Explanation

This verifies that the DNS server contains the zones required by the environment.

`benlab.local` provides forward DNS resolution for the Active Directory domain.

`20.20.10.in-addr.arpa` provides reverse DNS resolution for the `10.20.20.0/24` network.

`_msdcs.benlab.local` contains DNS information used extensively by Active Directory for Domain Controller and service discovery.

`branchlab.local` is the conditional forwarder configured earlier in the DNS module.

Result

Successful — required DNS zones were present.

Step 3 – Inspect DNS Resource Records

Review the resource records stored inside the `benlab.local` DNS zone.

Command

```
Get-DnsServerResourceRecord -ZoneName "benlab.local"
```

The output displayed several DNS record types, including:

A

AAAA

CNAME

NS

SOA

SRV

Records for systems such as DC01, FILESRV01, APP01, and WIN01 were present.

Active Directory SRV records for LDAP, Kerberos, and other domain services were also visible.

Explanation

DNS contains more than simple hostname-to-IP mappings.

Active Directory relies heavily on **SRV (Service) records** to advertise the location of services such as LDAP and Kerberos.

Result

Successful — host and Active Directory service records were present.

Step 4 – Verify WIN01 DNS Client Configuration

On WIN01, inspect the workstation's TCP/IP and DNS configuration.

Command

```
ipconfig /all
```

Important values observed:

```
IPv4 Address       : 10.20.20.140
Subnet Mask        : 255.255.255.0
Default Gateway    : 10.20.20.254
DHCP Server        : 10.20.20.254
DNS Server         : 10.20.20.10
DNS Suffix         : benlab.local
```

Explanation

This was particularly important because an Active Directory client should use the internal DNS server that understands the AD domain.

WIN01 was correctly configured to use:

```
10.20.20.10 → DC01
```

rather than using a public DNS server directly.

Result

Successful — WIN01 was configured to use DC01 for DNS resolution.

Step 5 – Test Forward DNS Resolution

From WIN01, test whether the hostname of FILESRV01 resolves to its IP address.

Command

```
Resolve-DnsName FILESRV01.benlab.local
```

Result

```
FILESRV01.benlab.local → 10.20.20.50
```

Explanation

This tests a **forward lookup**:

Hostname

↓

DNS

↓

IP Address

Result

Successful — FILESRV01 resolved to 10.20.20.50.

Step 6 – Test Reverse DNS Resolution

Test whether the IP address of FILESRV01 resolves back to its hostname.

Command

```
Resolve-DnsName 10.20.20.50
```

Result

```
10.20.20.50 → FILESRV01.benlab.local
```

The response was provided by a PTR record in the reverse lookup zone.

Explanation

This tests a **reverse lookup**:

IP Address

↓

PTR Record

↓

Hostname

Result

Successful — forward and reverse resolution were both functioning.

Step 7 – Verify LDAP SRV Record Discovery

Test whether WIN01 can use DNS to locate the LDAP service for the Active Directory domain.

Command

```
Resolve-DnsName _ldap._tcp.dc._msdcs.benlab.local -Type SRV
```

Important result:

```
NameTarget : dc01.benlab.local
```

```
Port       : 389
```

Explanation

This tells WIN01:

For an LDAP service associated with a Domain Controller in `benlab.local`, contact DC01 on port 389.

LDAP is used to access and query directory information such as users, groups, computers, and other Active Directory objects.

Result

Successful — LDAP service discovery through DNS worked.

Step 8 – Verify Kerberos SRV Record Discovery

Test Kerberos service discovery.

Command

```
Resolve-DnsName _kerberos._tcp.dc._msdcs.benlab.local -Type SRV
```

Important result:

```
NameTarget : dc01.benlab.local
```

```
Port       : 88
```

Explanation

Kerberos is the primary authentication protocol used by Active Directory.

The SRV record tells domain clients which Domain Controller provides Kerberos authentication services.

Result

Successful — DC01 was discovered as a Kerberos service provider on port 88.

Step 9 – Test Kerberos and LDAP Port Connectivity

DNS service discovery proves that the appropriate SRV records exist, but it does not by itself prove that the client can establish TCP connections to those service ports.

Test Kerberos:

```
Test-NetConnection dc01.benlab.local -Port 88
```

Result:

```
TcpTestSucceeded : True
```

Test LDAP:

```
Test-NetConnection dc01.benlab.local -Port 389
```

Result:

```
TcpTestSucceeded : True
```

Explanation

This demonstrated an important troubleshooting distinction:

Resolve-DnsName

↓

Can I FIND the service?

Test-NetConnection

↓

Can I REACH the service's TCP port?

Result

Successful — WIN01 could reach both Kerberos and LDAP service ports on DC01.

Step 10 – Verify Domain Controller Discovery

Test whether WIN01 can locate a Domain Controller for the domain.

Command

```
nltest /dsgetdc:benlab.local
```

The output identified:

```
DC: \\DC01.benlab.local
```

```
Address: \\10.20.20.10
```

```
Dom Name: benlab.local
```

```
Forest Name: benlab.local
```

```
Our Site Name: Default-First-Site-Name
```

The command completed successfully

Explanation

This goes beyond simply resolving `dc01.benlab.local`.

It verifies that WIN01 can actually **discover a Domain Controller** for the Active Directory domain.

Result

Successful — WIN01 successfully discovered DC01.

Step 11 – Test External DNS Resolution

Test whether WIN01 can resolve an Internet hostname.

Command

```
Resolve-DnsName google.com
```

Public IPv4 and IPv6 addresses were successfully returned.

Explanation

WIN01 is configured to use DC01 as its DNS server.

Therefore, external resolution demonstrates the following path:

WIN01

|
| DNS query



DC01

|
| Forwarding / recursion



External DNS

|



google.com resolved

Result

Successful — external DNS resolution worked.

Step 12 – Test External TCP Connectivity

Test HTTPS connectivity separately from DNS resolution.

Command

```
Test-NetConnection google.com -Port 443
```

Result:

```
RemotePort          : 443
```

```
TcpTestSucceeded    : True
```

Explanation

DNS resolution and network connectivity are related but separate troubleshooting layers.

A hostname can resolve successfully while the destination service remains unreachable.

Result

Successful — WIN01 had external HTTPS connectivity.

Step 13 – Inspect the DNS Client Cache

View locally cached DNS records on WIN01.

Command

```
Get-DnsClientCache
```

The output displayed DNS records previously resolved by WIN01.

Explanation

Windows caches DNS responses locally to avoid repeatedly querying the DNS server for the same records.

Each cached record remains available according to its **TTL (Time To Live)**.

Result

DNS client cache successfully inspected.

Step 14 – Clear and Repopulate the DNS Client Cache

Clear WIN01's DNS client cache.

Command

```
Clear-DnsClientCache
```

Then perform a fresh lookup:

```
Resolve-DnsName dc01.benlab.local
```

Finally, inspect the cache again:

Get-DnsClientCache

DC01 appeared in the cache after the new lookup.

Explanation

This demonstrated the troubleshooting process:

Cached DNS information exists

↓

Clear-DnsClientCache

↓

Cached entries removed

↓

Resolve-DnsName

↓

Fresh DNS query

↓

Response cached again

This is particularly useful when troubleshooting stale or incorrect DNS information.

Result

Successful — DNS cache clearing and repopulation were demonstrated.

Step 15 – Perform Final DNS Health Diagnostic

Finally, run an Active Directory DNS diagnostic directly from DC01.

Command

```
dcdiag /test:dns
```

Important results:

DC01 passed test Connectivity

DC01 passed test DNS

benlab.local passed test DNS

Explanation

`dcdiag` is a Domain Controller diagnostic utility.

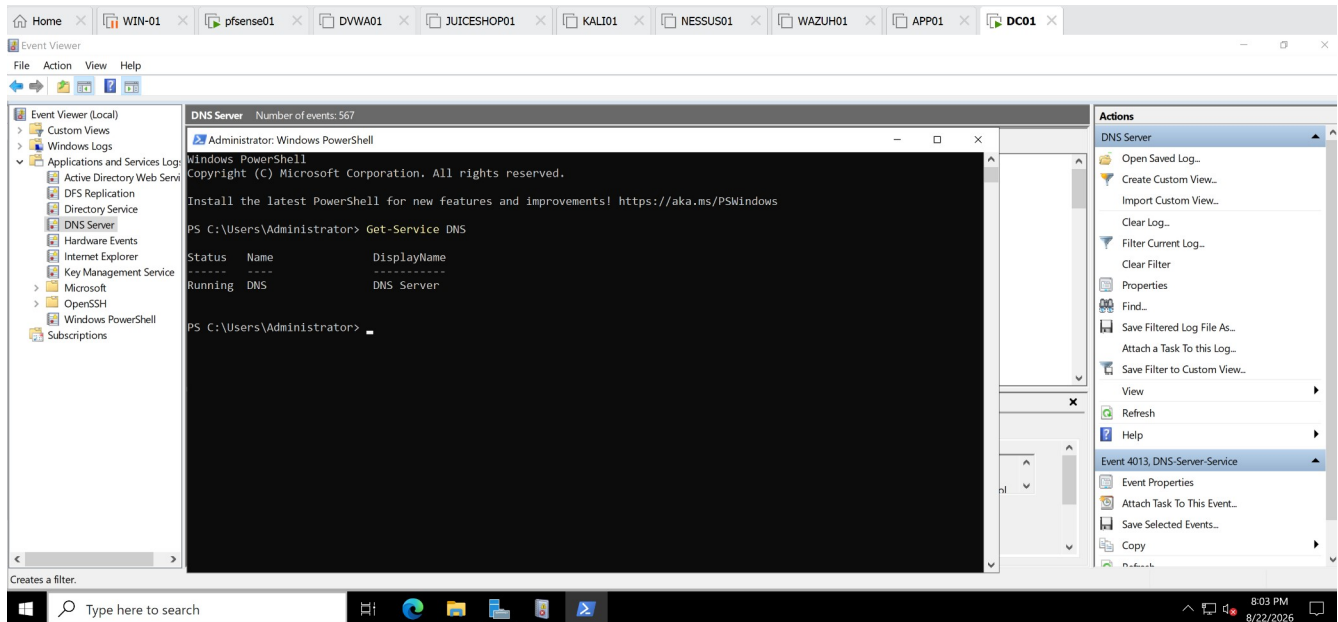
The DNS test examines DNS functionality required by Active Directory and provides a strong final server-side validation after client-side troubleshooting has been completed.

Result

Successful — DC01 and benlab.local passed the DNS diagnostic.

Screenshots

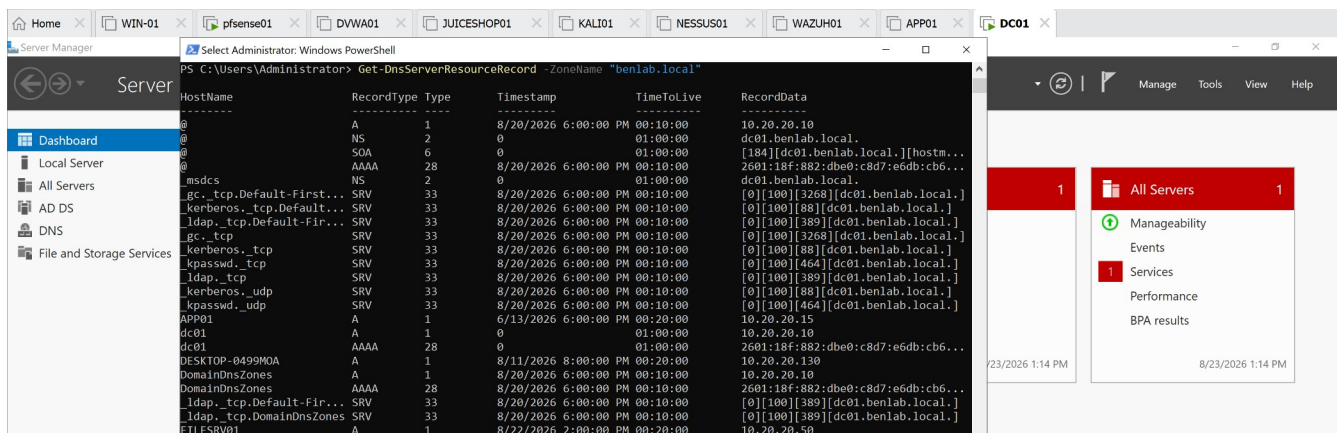
This section provides visual evidence of the DNS troubleshooting and validation performed during Task 5.



Screenshot 1 – DNS Service and Zone Validation

Caption:

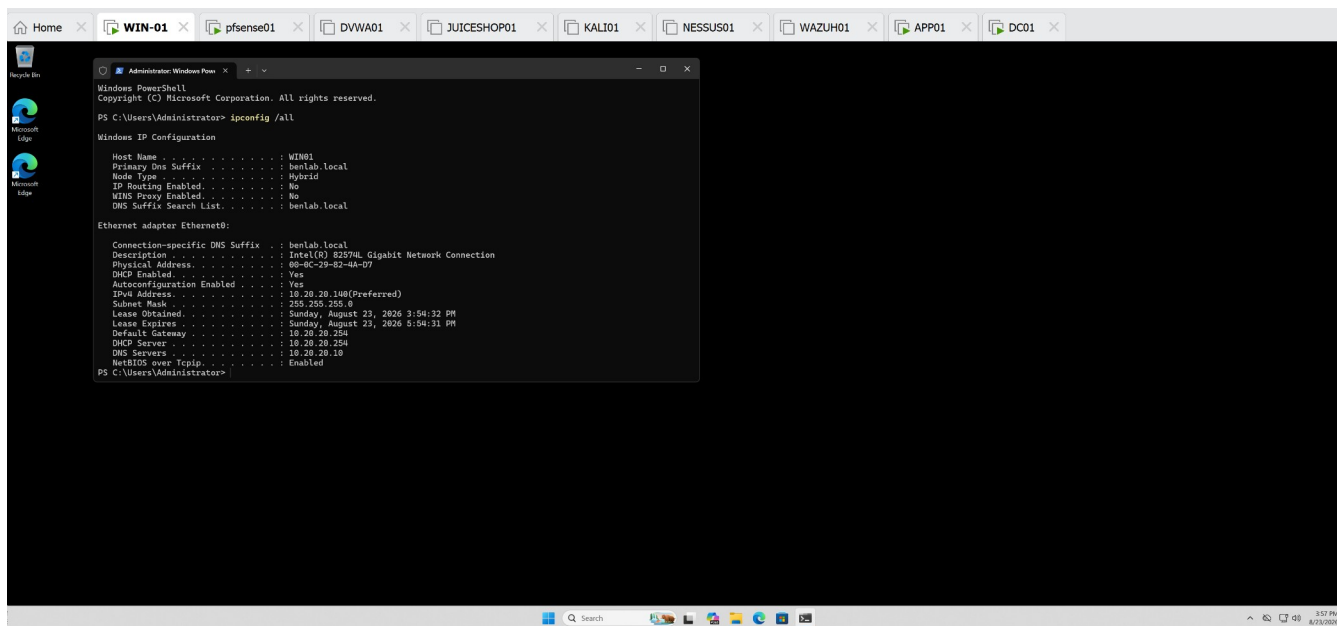
DNS Server service and DNS zone validation on DC01, confirming that the DNS service is running and the required Active Directory DNS zones are available.



Screenshot 2 – DNS Resource Record

Caption:

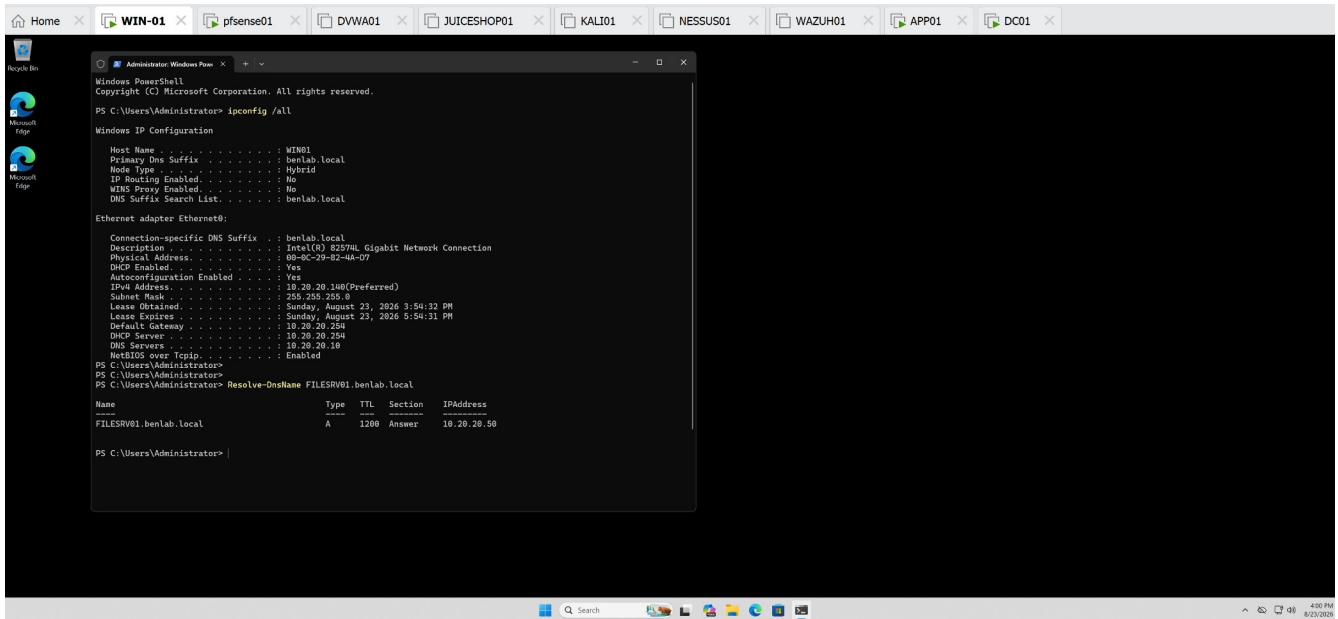
DNS resource record inspection for `benlab.local`, showing host records and Active Directory SRV records used for service discovery.



Screenshot 3 – WIN01 DNS Client Configuration

Caption:

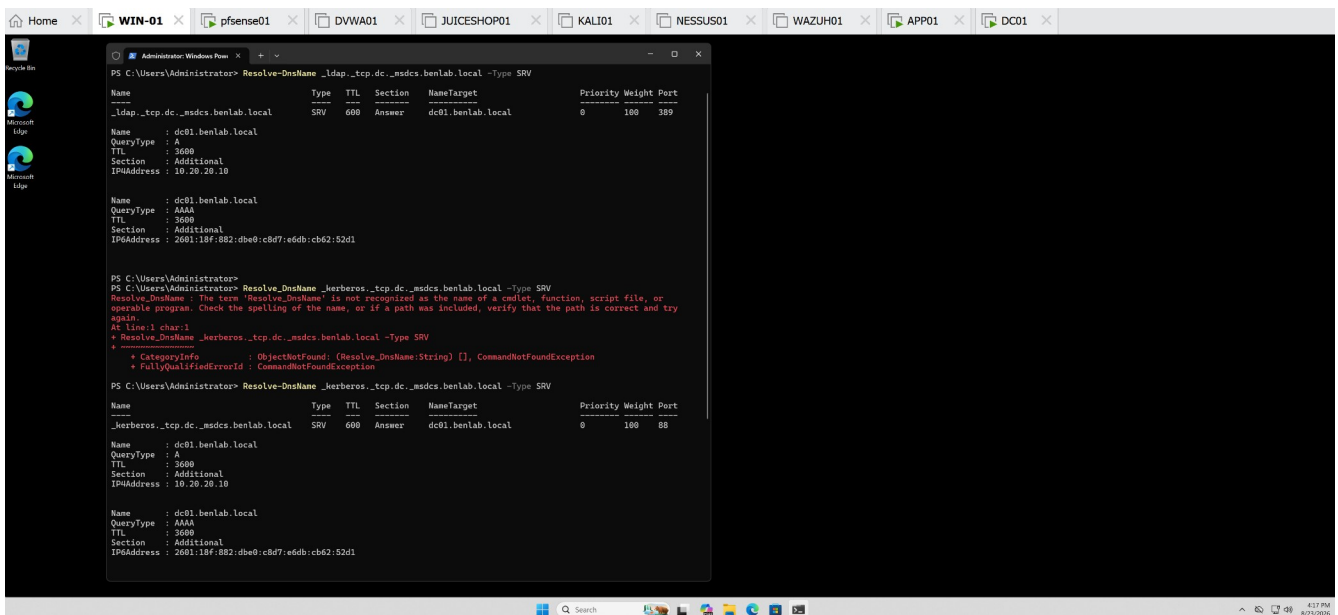
WIN01 TCP/IP configuration confirming that the domain workstation receives network configuration from pfSense and uses DC01 (10.20.20.10) as its DNS server.



Screenshot 4 – Forward and Reverse DNS Resolution

Caption:

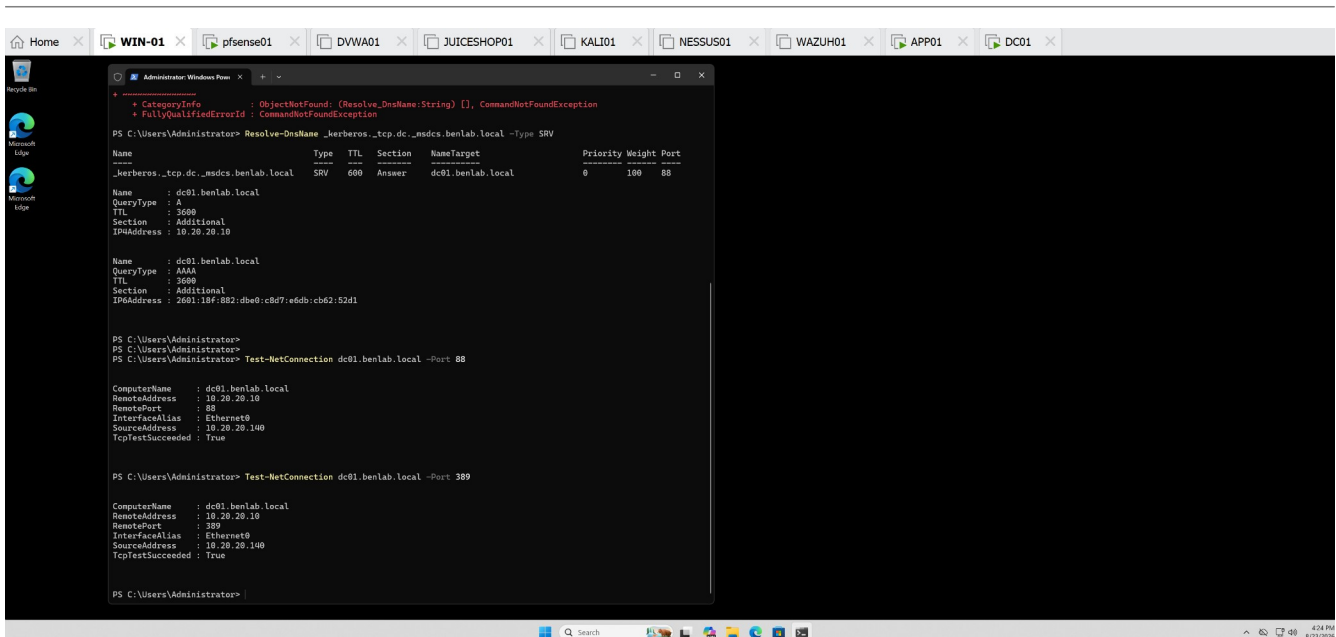
Successful forward and reverse DNS resolution for FILESRV01, validating both the A record and PTR record.



Screenshot 5 – LDAP and Kerberos SRV Discovery

Caption:

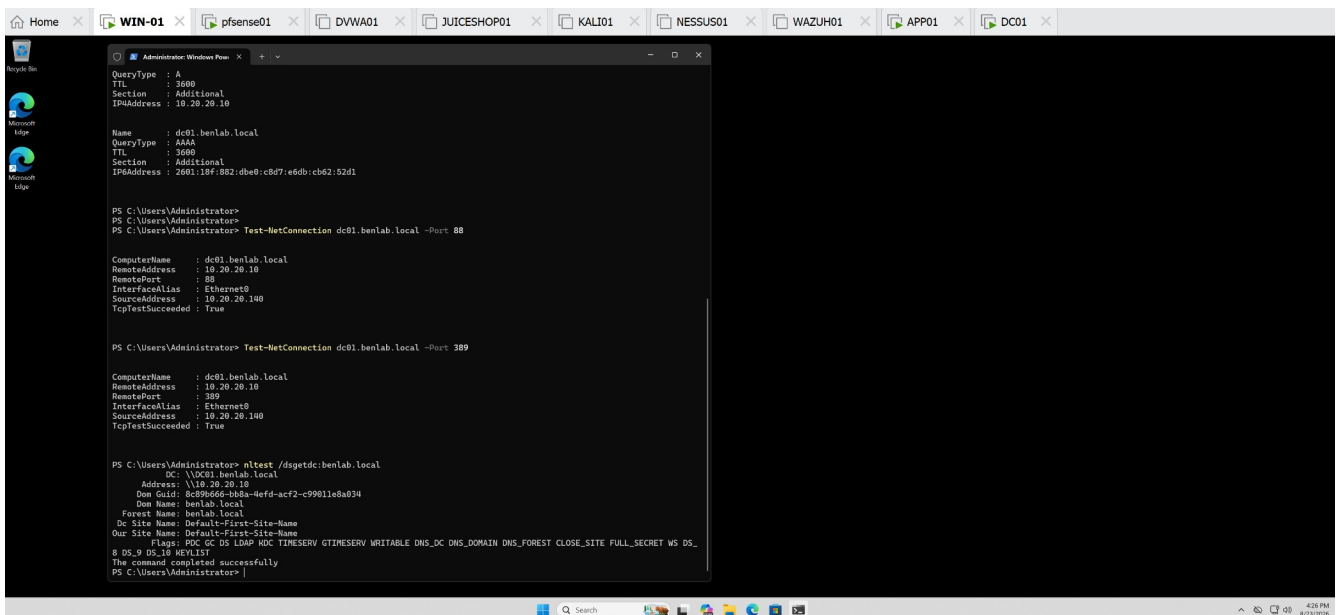
Active Directory SRV record validation demonstrating successful discovery of LDAP and Kerberos services hosted by DC01.



Screenshot 6 – Kerberos and LDAP Connectivity

Caption:

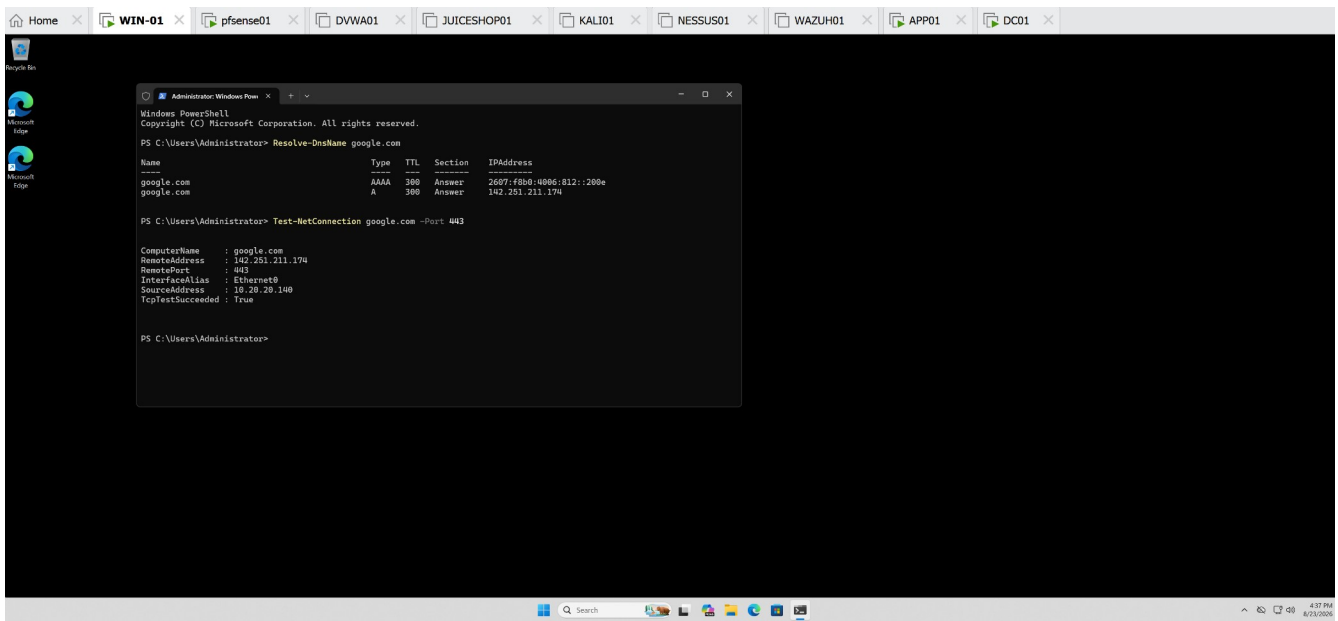
Successful TCP connectivity from WIN01 to DC01 over Kerberos port 88 and LDAP port 389.



Screenshot 7 – Domain Controller Discovery

Caption:

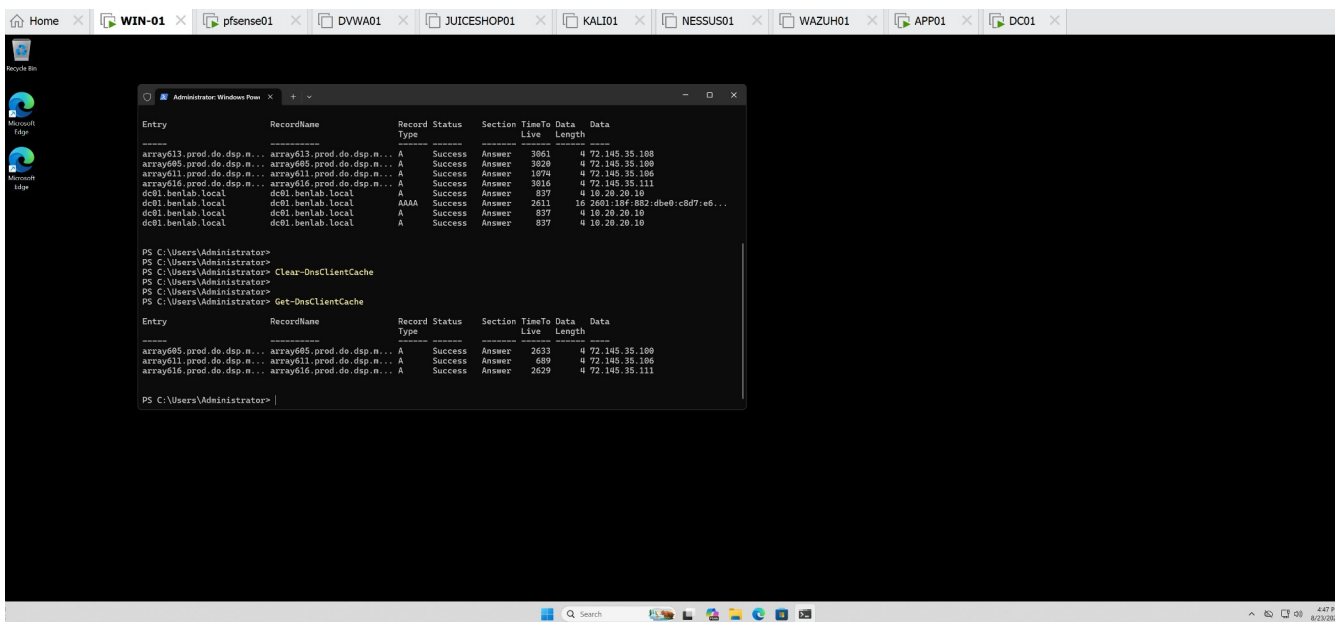
Successful Active Directory Domain Controller discovery from WIN01, identifying DC01 (10.20.20.10) as a Domain Controller for benlab.local.



Screenshot 8 – External DNS and Connectivity

Caption:

Successful external DNS resolution and HTTPS connectivity from WIN01, validating external name resolution through the internal DNS infrastructure.

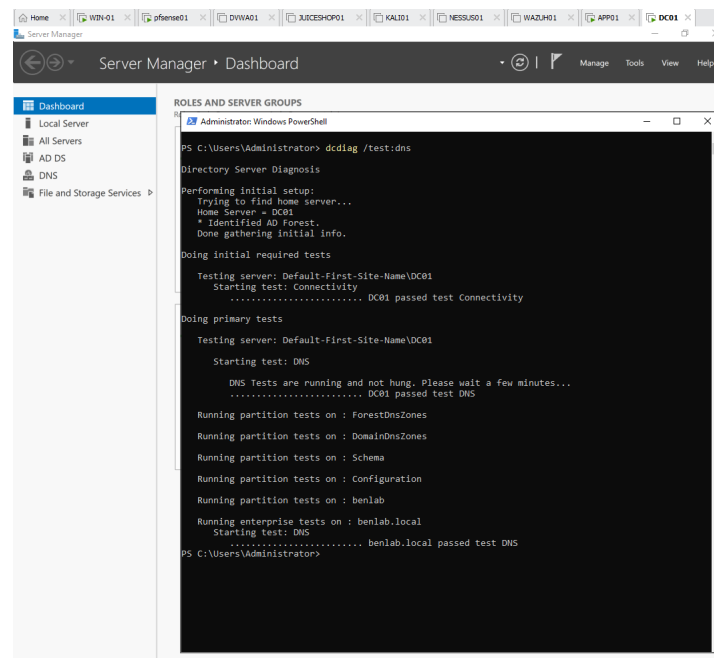


Screenshot 9 – DNS Client Cache

Caption:

DNS client cache troubleshooting on WIN01, demonstrating cache inspection, clearing, fresh name resolution, and cache repopulation.

Screenshot 10 – Final DNS Diagnostic



Caption:

Final Active Directory DNS diagnostic on DC01 confirming successful connectivity and DNS health for the `benlab.local` domain.

Troubleshooting Sequence Learned

A systematic troubleshooting sequence was developed during this task:

1. Verify client IP configuration

↓

2. Verify configured DNS server

↓

3. Verify DNS Server service

↓

4. Verify DNS zones

↓

5. Inspect DNS records

↓

6. Test forward resolution

↓

7. Test reverse resolution

- ↓
8. Verify AD SRV records
- ↓
9. Test LDAP/Kerberos connectivity
- ↓
10. Verify Domain Controller discovery
- ↓
11. Test external DNS resolution
- ↓
12. Test external connectivity
- ↓
13. Inspect/clear DNS cache
- ↓
14. Run `dcdiag /test:dns`

The main lesson is to troubleshoot DNS **layer by layer instead of immediately changing configurations**.

Key Learning Points

- DNS is a critical component of Active Directory infrastructure.
- Domain clients should use the internal Active Directory DNS server for domain name resolution.
- **A records** map hostnames to IPv4 addresses.
- **PTR records** provide reverse IP-to-hostname resolution.
- **SRV records** advertise network services and allow Active Directory clients to locate Domain Controllers and services.
- **LDAP**, primarily using TCP port **389**, provides access to Active Directory directory information.
- **Kerberos**, using port **88**, provides Active Directory authentication.
- Successful DNS resolution does not automatically prove that the associated network service is reachable.
- `Test-NetConnection` can verify TCP connectivity to specific service ports.
- `nltest /dsgetdc:` can verify Domain Controller discovery from a domain workstation.

- `Get-DnsClientCache` allows administrators to inspect cached DNS responses.
 - `Clear-DnsClientCache` is useful when troubleshooting stale DNS information.
 - `Resolve-DnsName` can generate fresh DNS queries and test specific DNS record types.
 - `dcdiag /test:dns` provides a comprehensive DNS diagnostic from the Domain Controller perspective.
 - Effective troubleshooting should follow a logical sequence from **network configuration** → **DNS infrastructure** → **name resolution** → **AD service discovery** → **connectivity** → **final validation**.
-

Commands Used

```
Get-Service DNS
```

```
Get-DnsServerZone
```

```
Get-DnsServerResourceRecord -ZoneName "benlab.local"
```

```
ipconfig /all
```

```
Resolve-DnsName FILESRV01.benlab.local
```

```
Resolve-DnsName 10.20.20.50
```

```
Resolve-DnsName _ldap._tcp.dc._msdcs.benlab.local -Type SRV
```

```
Resolve-DnsName _kerberos._tcp.dc._msdcs.benlab.local -Type SRV
```

```
Test-NetConnection dc01.benlab.local -Port 88
```

```
Test-NetConnection dc01.benlab.local -Port 389
```

```
nltest /dsgetdc:benlab.local
```

```
Resolve-DnsName google.com
```

```
Test-NetConnection google.com -Port 443
```

```
Get-DnsClientCache
```

```
Clear-DnsClientCache
```

```
Resolve-DnsName dc01.benlab.local
```

```
dcdiag /test:dns
```

Final Result

Task 5 – DNS Troubleshooting and Validation: COMPLETED SUCCESSFULLY

The `benlab.local` DNS infrastructure was validated from both the **client and server perspectives**.

WIN01 successfully:

- Used DC01 as its DNS server.
- Resolved internal hostnames.
- Performed reverse DNS lookups.
- Discovered LDAP and Kerberos services through SRV records.
- Connected to LDAP TCP 389 and Kerberos TCP 88.
- Located the `benlab.local` Domain Controller.
- Resolved external Internet hostnames.
- Established external HTTPS connectivity.
- Cleared and repopulated its DNS client cache.

DC01 successfully passed:

```
dcdiag /test:dns
```

confirming healthy DNS functionality for the Active Directory environment.
